

# СОДЕРЖАНИЕ

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ.....                                                                           | 3  |
| Глава 1. Теоретические основы рекомендательных систем и анализ предметной области ..... | 8  |
| 1.1. Современные рекомендательные системы.....                                          | 8  |
| 1.2. Основные подходы к построению рекомендательных систем .                            | 9  |
| 1.2.1. Collaborative Filtering .....                                                    | 9  |
| 1.2.2. Content-Based Filtering .....                                                    | 10 |
| 1.2.3. Гибридные рекомендательные системы.....                                          | 11 |
| 1.3. Рекомендательные системы в игровой индустрии.....                                  | 12 |
| 1.4. Особенности задач рекомендаций в Dota 2.....                                       | 12 |
| 1.4.1. Высокая комбинаторная сложность.....                                             | 13 |
| 1.4.2. Синергия и контрпики .....                                                       | 13 |
| 1.4.3. Ролевая зависимость .....                                                        | 14 |
| 1.4.4. Изменение игровой меты .....                                                     | 14 |
| 1.5. Методы машинного обучения для ranking-задач.....                                   | 14 |
| 1.5.1. Gradient Boosting .....                                                          | 14 |
| 1.5.2. Learning-to-Rank .....                                                           | 15 |
| 1.5.3. LightGBM Ranker .....                                                            | 15 |
| 1.6. Графовые методы анализа данных.....                                                | 15 |
| 1.6.1. PageRank .....                                                                   | 16 |
| 1.6.2. Графовые признаки.....                                                           | 16 |
| 1.7. Embedding-представления игровых персонажей .....                                   | 16 |
| 1.8. Выводы по главе .....                                                              | 17 |
| Глава 2. Сбор и анализ данных для рекомендательной системы.....                         | 18 |
| 2.1. Источники данных и структура игрового датасета .....                               | 18 |

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 2.2. Архитектура сбора и обработки данных .....                            | 23 |
| 2.3. Статистические признаки.....                                          | 27 |
| 2.3.1. Матрица синергий.....                                               | 29 |
| 2.3.2. Матрица контрпиков .....                                            | 32 |
| 2.3.4. Графовые признаки.....                                              | 36 |
| 2.3.5. Embedding-представления персонажей .....                            | 38 |
| 2.3.6. Hard Negative Sampling.....                                         | 42 |
| 2.3.7. Формирование итогового признакового пространства.....               | 45 |
| Выводы по главе 2 .....                                                    | 47 |
| Глава 3. Разработка и тестирование рекомендательной системы.....           | 49 |
| 3.1 Архитектура рекомендательной системы .....                             | 49 |
| 3.2 Обучение ranking-модели на основе LightGBM .....                       | 51 |
| 3.4. Оценка качества ranking-модели .....                                  | 55 |
| 3.5. Разработка пользовательского веб-интерфейса.....                      | 58 |
| ЗАКЛЮЧЕНИЕ .....                                                           | 64 |
| СПИСОК ЛИТЕРАТУРЫ (ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ) И<br>ИНТЕРНЕТ-РЕСУРСОВ ..... | 67 |

## ВВЕДЕНИЕ

Современная индустрия компьютерных игр является одной из наиболее быстроразвивающихся областей цифровых технологий, активно использующей методы анализа данных, машинного обучения и интеллектуальных рекомендательных систем. Особенно значимую роль аналитические системы приобретают в многопользовательских соревновательных играх жанра Multiplayer Online Battle Arena (MOBA), где результат игрового матча зависит не только от индивидуальных навыков игроков, но и от корректного подбора игровых персонажей, их взаимодействия между собой, распределения ролей и способности противодействовать выбранным персонажам соперника.

Одной из наиболее популярных дисциплин жанра MOBA является игра Dota 2, разработанная компанией Valve Corporation. Игровой процесс Dota 2 характеризуется высокой степенью вариативности, сложной системой взаимодействий между персонажами и постоянными изменениями игрового баланса. На текущий момент в игре представлено более 120 уникальных персонажей, каждый из которых обладает собственным набором способностей, характеристик, игровых ролей и особенностей взаимодействия с другими героями. В рамках одного матча игроки формируют так называемый «драфт» — последовательность выбора персонажей для своей команды с учетом уже выбранных союзников и противников.

Процесс выбора персонажей представляет собой сложную многокритериальную задачу принятия решений в условиях высокой неопределенности. Игроку необходимо одновременно учитывать такие показатели как синергию персонажа с союзниками, эффективность против персонажей соперника, соответствие выбранной игровой роли, текущую игровую мету, статистические показатели побед, особенности конкретной игровой ситуации и т.д.

Количество возможных комбинаций персонажей в рамках драфта является чрезвычайно большим, что приводит к комбинаторному росту пространства состояний. В результате даже опытные игроки испытывают сложности при выборе оптимального персонажа в условиях ограниченного времени принятия решения.

Существующие игровые аналитические платформы и рекомендательные сервисы преимущественно используют упрощенные эвристические методы, основанные на глобальных статистиках побед, популярных сочетаниях персонажей или заранее заданных правилах. Подобные подходы обладают рядом существенных ограничений: недостаточный учет контекста текущего драфта, слабая адаптация к изменению игровой меты, отсутствие комплексного анализа взаимодействий между персонажами, ограниченная персонализация рекомендаций, низкая интерпретируемость результатов.

В связи с этим возникает необходимость разработки интеллектуальной рекомендательной системы, способной анализировать игровые данные, учитывать сложные зависимости между персонажами и формировать рекомендации на основе методов машинного обучения и анализа графовых структур.

Использование методов машинного обучения в задачах подбора игровых персонажей позволяет перейти от статических эвристических рекомендаций к динамическим моделям ранжирования, способным учитывать множество признаков одновременно. Особую актуальность в данной области приобретают методы gradient boosting, learning-to-rank, graph analytics и embedding-представления объектов, позволяющие выявлять скрытые зависимости между игровыми сущностями и строить более точные модели рекомендаций.

Дополнительную сложность представляет необходимость обработки больших объемов игровых данных, получаемых из внешних API и статистических платформ. Для построения качественной

рекомендательной системы требуется реализация полноценного pipeline обработки данных, включающего: сбор и агрегацию игровых статистик, очистку и предобработку данных, генерацию признаков, построение графовых характеристик, формирование embedding-представлений;, обучение и оценку моделей машинного обучения, реализацию inference-механизма рекомендаций.

Таким образом, разработка интеллектуальной рекомендательной системы для подбора игровых персонажей является актуальной научно-прикладной задачей, находящейся на пересечении областей машинного обучения, анализа данных, рекомендательных систем и игровой аналитики.

Целью выпускной квалификационной работы является разработка гибридной рекомендательной системы для подбора игровых персонажей в Dota 2 на основе анализа игровых данных, методов машинного обучения и графовых характеристик взаимодействия персонажей.

Для достижения поставленной цели были сформулированы следующие задачи:

- Провести анализ предметной области и существующих подходов к построению рекомендательных систем.
- Исследовать особенности задач рекомендаций в многопользовательских МОБА-играх.
- Выполнить сбор, очистку и предобработку игровых данных.
- Разработать pipeline генерации признаков для анализа игровых персонажей.
- Построить граф взаимодействий персонажей и вычислить графовые признаки.
- Реализовать embedding-представления персонажей для выявления скрытых зависимостей между игровыми сущностями.

- Разработать модель ранжирования персонажей на основе алгоритмов gradient boosting.
- Реализовать механизм hard negative sampling для повышения качества обучения модели.
- Провести обучение, тестирование и сравнительный анализ моделей машинного обучения.
- Разработать пользовательский интерфейс рекомендательной системы.
- Провести оценку качества рекомендаций с использованием современных метрик машинного обучения.

Объектом исследования являются рекомендательные системы в задачах подбора игровых персонажей в многопользовательских соревновательных играх.

Предметом исследования являются методы машинного обучения, алгоритмы ранжирования, графовые модели и embedding-подходы, применяемые для анализа игровых данных и формирования рекомендаций персонажей.

Научная новизна работы заключается в разработке гибридного подхода к построению рекомендательной системы, объединяющего: методы learning-to-rank, графовый анализ взаимодействий персонажей, embedding-представления игровых сущностей, контекстно-зависимое ранжирование персонажей, hard negative sampling для улучшения качества обучения модели.

В отличие от традиционных подходов, основанных исключительно на глобальных статистиках побед или фиксированных эвристиках, предложенная система учитывает структуру взаимодействий между персонажами, особенности текущего драфта и скрытые зависимости в игровом пространстве.

Практическая значимость работы заключается в создании программной системы, способной в реальном времени формировать рекомендации по выбору персонажей на основе текущего состояния игрового драфта. Разработанное решение может быть использовано: игроками для повышения эффективности выбора персонажей, аналитическими игровыми платформами, киберспортивными командами, исследователями в области game analytics, разработчиками интеллектуальных рекомендательных систем.

Дополнительно результаты работы могут быть использованы для дальнейших исследований в области recommendation systems, graph machine learning, ranking systems и интеллектуального анализа игровых данных.

В рамках работы были использованы язык программирования Python, библиотеки Pandas, Scikit-learn, LightGBM и Streamlit, а также данные, полученные посредством OpenDota API. Архитектура системы включает этапы ETL-обработки данных, генерации признаков, построения embedding-представлений, вычисления графовых характеристик и inference-механизма ранжирования персонажей.

Структура выпускной квалификационной работы включает введение, три главы, заключение, список использованных источников и приложения. В первой главе рассматриваются теоретические основы рекомендательных систем и анализ предметной области. Во второй главе описываются архитектура системы, методы обработки данных и реализация моделей машинного обучения. В третьей главе представлены результаты экспериментов, анализ качества модели и оценка эффективности разработанного решения.

# Глава 1. Теоретические основы рекомендательных систем и анализ предметной области

## 1.1. Современные рекомендательные системы

В условиях стремительного роста объёмов цифровой информации и увеличения количества пользовательского контента особую актуальность приобретают интеллектуальные рекомендательные системы (Recommendation Systems), предназначенные для автоматизированного подбора наиболее релевантных объектов для пользователя. Подобные системы широко применяются в электронной коммерции, мультимедийных сервисах, социальных сетях, стриминговых платформах, онлайн-образовании, киберспорте и игровой аналитике.

Основной задачей рекомендательной системы является прогнозирование предпочтений пользователя и формирование персонализированного списка объектов, обладающих наибольшей вероятностью заинтересовать пользователя в конкретном контексте. В качестве рекомендуемых объектов могут выступать товары, фильмы, музыкальные композиции, новости, игровые стратегии, персонажи, игровые предметы и другие сущности предметной области.

С точки зрения машинного обучения рекомендательная система представляет собой задачу построения функции:

$$f(u, i, c) \rightarrow r$$

*Формула 1.1 — Формальная модель рекомендательной системы*

где:

$u$  — пользователь;

$i$  — объект рекомендации;

$c$  — контекст взаимодействия;

$r$  — прогнозируемая релевантность объекта.

Современные рекомендательные системы являются важным компонентом интеллектуального анализа данных, поскольку позволяют существенно



сократить информационную перегрузку пользователя и повысить качество взаимодействия с цифровыми сервисами.

В настоящее время рекомендательные системы активно применяются в игровой индустрии. Игровые платформы используют методы анализа данных для: персонализации игрового опыта, формирования рекомендаций игровых персонажей, анализа игровых стратегий, matchmaking, прогнозирования поведения игроков, балансировки игровых механик.

Особенно актуальными подобные подходы являются в многопользовательских соревновательных играх, где корректность выбора игровых объектов напрямую влияет на результат матча.

## **1.2. Основные подходы к построению рекомендательных систем**

Современные рекомендательные системы можно разделить на несколько основных категорий: collaborative filtering, content-based filtering, hybrid recommendation systems, contextual recommendation systems, knowledge-based systems.

Каждый из подходов обладает собственными преимуществами и ограничениями.

### **1.2.1. Collaborative Filtering**

Collaborative filtering является одним из наиболее распространённых подходов к построению рекомендательных систем. Основная идея метода заключается в анализе поведения пользователей и поиске закономерностей во взаимодействиях между пользователями и объектами.

Метод основывается на предположении о том, что пользователи с похожими предпочтениями будут демонстрировать схожее поведение в будущем.

Collaborative filtering подразделяется на:

- user-based collaborative filtering;
- item-based collaborative filtering.

User-based подход предполагает поиск пользователей с похожими предпочтениями, после чего рекомендации формируются на основе действий схожих пользователей.

Item-based подход анализирует сходство объектов между собой и рекомендует объекты, похожие на ранее выбранные пользователем.

Для представления взаимодействий обычно используется матрица пользователь–объект:

$$R = (r_{ui})_{m \times n}$$

*Формула 1.2 — Матрица пользовательских предпочтений*

где:

$m$  — количество пользователей;

$n$  — количество объектов;

$r_{ui}$  — взаимодействие пользователя  $u$  с объектом  $i$ .

Для вычисления сходства между объектами или пользователями часто используется косинусное сходство:

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

*Формула 1.3 — Метрика cosine similarity для оценки сходства векторов*

Несмотря на высокую популярность collaborative filtering, данный подход обладает рядом ограничений: проблема холодного старта (cold start), высокая разреженность матриц взаимодействий, слабый учет контекста, недостаточная интерпретируемость результатов.

В задачах подбора игровых персонажей данные ограничения особенно критичны, поскольку игровые ситуации динамически изменяются в зависимости от текущего драфта.

#### 1.2.2. Content-Based Filtering

Content-based filtering основан на анализе характеристик объектов и формировании рекомендаций на основе их признакового описания.

В рамках данного подхода каждому объекту сопоставляется вектор признаков:

$$x_i = (x_{i1}, x_{i2}, \dots, x_{in})$$

*Формула 5 — Векторное представление объекта в признаковом пространстве*

В задачах игровой аналитики признаками могут выступать: роль персонажа, показатели побед, игровые характеристики, тип урона, мобильность, показатели синергии, показатели контрпигов.

Преимуществами content-based подхода являются: отсутствие зависимости от большого количества пользовательских данных, высокая интерпретируемость, возможность учета предметных особенностей объектов.

Однако подобные системы часто демонстрируют ограниченное разнообразие рекомендаций и плохо выявляют скрытые зависимости между объектами.

#### 1.2.3. Гибридные рекомендательные системы

Наиболее эффективным современным подходом являются гибридные рекомендательные системы (Hybrid Recommendation Systems), объединяющие преимущества нескольких методов одновременно. Гибридные системы позволяют: учитывать контекст взаимодействия, анализировать признаки объектов, использовать статистические зависимости, объединять различные модели машинного обучения, повышать устойчивость рекомендаций.

В рамках данной работы используется гибридный подход, сочетающий:

- feature engineering;
- graph analytics;
- embedding representations;
- learning-to-rank;
- gradient boosting.

Подобная архитектура позволяет учитывать как статистические характеристики персонажей, так и скрытые зависимости между ними.

### **1.3. Рекомендательные системы в игровой индустрии**

Игровая индустрия является одной из наиболее перспективных областей применения методов машинного обучения и интеллектуального анализа данных.

Современные игровые платформы генерируют огромные объемы информации: игровые события, действия пользователей, статистики матчей, временные ряды, социальные взаимодействия, данные игровых стратегий.

В связи с этим активно развиваются такие направления как:

game analytics

esports analytics

behavioral analytics

matchmaking systems

recommendation systems.

В многопользовательских играх рекомендательные системы используются для: подбора игровых предметов, формирования стратегий, анализа сборок персонажей, подбора союзников, анализа игровых ролей, формирования рекомендаций по выбору персонажей.

Особую сложность представляют МОБА-игры, характеризующиеся высокой динамикой игрового процесса и сложными зависимостями между игровыми сущностями.

### **1.4. Особенности задач рекомендаций в Dota 2**

Dota 2 представляет собой многопользовательскую соревновательную игру жанра МОБА, в которой две команды по пять игроков соревнуются друг с другом, управляя уникальными игровыми персонажами.

Одной из ключевых стадий матча является этап драфта — последовательный выбор персонажей игроками.

Корректность выбора персонажей оказывает существенное влияние на следующие переменные: вероятность победы, распределение ролей, эффективность командных взаимодействий, контроль карты, темп игры.

Задача выбора персонажа осложняется несколькими факторами.

#### 1.4.1. Высокая комбинаторная сложность

На момент выполнения работы игра содержит более 120(129) уникальных персонажей. Количество возможных комбинаций выбора героев является чрезвычайно большим.

Количество потенциальных комбинаций можно оценить следующим образом:

$$C_{nk} = \frac{n!}{k! (n - k)!}$$

*Формула 6 — Вычисление количества возможных комбинаций*

Получаем примерно  $8.9 \times 10^{20}$

Полный перебор всех вариантов становится вычислительно невозможным.

#### 1.4.2. Синергия и контрпики

Важной особенностью Dota 2 является наличие сложных взаимодействий между персонажами.

Некоторые герои эффективно усиливают друг друга: совместные способности, усиление мобильности, комбинирование контроля, синергия ультимативных способностей.

Другие персонажи, напротив, выступают контрпиками: блокируют способности, нейтрализуют мобильность, снижают эффективность противника или даже совсем “выключают” его из игры.

Таким образом, задача рекомендации персонажа является контекстно-зависимой и не может решаться исключительно на основе глобальных статистик побед.

#### 1.4.3. Ролевая зависимость

Каждый игрок в команде выполняет определенную игровую роль: Carry, Mid, Offlane, Soft Support, Hard Support.

Эффективность персонажа существенно зависит от соответствия выбранной роли текущему состоянию команды.

Поэтому рекомендательная система должна учитывать: баланс ролей, игровые позиции, совместимость ролей внутри команды.

#### 1.4.4. Изменение игровой меты

Игровой баланс Dota 2 регулярно изменяется разработчиками посредством обновлений (patches). Изменения могут затрагивать: способности персонажей, игровые предметы, экономику, механику игры.

В результате актуальность персонажей динамически изменяется, что требует регулярного обновления статистических данных и переобучения моделей.

### 1.5. Методы машинного обучения для ranking-задач

В классических задачах рекомендательных систем широко применяются такие методы как: классификации, регрессии, ранжирования.

Для задач подбора игровых персонажей наиболее эффективным является подход learning-to-rank, поскольку система должна не просто предсказать вероятность успеха, а упорядочить кандидатов по степени релевантности а конкретный момент.

#### 1.5.1. Gradient Boosting

Одним из наиболее эффективных методов решения ranking-задач является gradient boosting.

Gradient boosting представляет собой ансамблевый метод машинного обучения, основанный на последовательном построении слабых моделей.

Итоговая модель формируется следующим образом:

$$Fm(x) = F_m - 1(x) + \gamma_m h_m(x)$$

*Формула 7 — Итеративное обновление модели в градиентном бустинге*

где:

$Fm(x)$  — итоговая модель;

$hm(x)$  — слабый алгоритм;

$\gamma_m$  — коэффициент вклада модели.

Преимуществами gradient boosting являются: высокая точность, устойчивость к шуму, работа с нелинейными зависимостями, поддержка heterogeneous features.

#### 1.5.2. Learning-to-Rank

Learning-to-rank представляет собой подход машинного обучения, ориентированный на задачи ранжирования объектов. Основной задачей модели является формирование корректного порядка объектов по степени релевантности. В данной работе используется pairwise ranking подход, минимизирующий функцию потерь:

$$L = -\sum(i, j) \log \sigma(s_i - s_j)$$

*Формула 8 — Функция потерь для задачи ранжирования*

где:

$s_i$  — оценка релевантности положительного объекта;

$s_j$  — оценка отрицательного объекта.

Подобный подход позволяет эффективно обучать модель различать хорошие и плохие варианты выбора персонажей.

#### 1.5.3. LightGBM Ranker

В качестве основной модели в работе используется алгоритм LightGBM Ranker. LightGBM является высокопроизводительной реализацией gradient boosting, оптимизированной для работы с большими объемами данных. Ключевые преимущества LightGBM: высокая скорость обучения, низкое потребление памяти, поддержка ranking-задач, эффективная работа с категориальными признаками, высокая масштабируемость.

Использование LightGBM Ranker позволяет учитывать большое количество признаков и сложные нелинейные зависимости между игровыми объектами.

### 1.6. Графовые методы анализа данных

Во многих задачах recommendation systems объекты могут быть представлены в виде графов.

В рамках данной работы персонажи и их взаимодействия моделируются в виде графовой структуры: вершины графа — игровые персонажи, ребра — показатели синергии или противодействия.

Подобное представление позволяет анализировать: структуру взаимодействий, влияние персонажей, плотность связей, скрытые сообщества.

#### 1.6.1. PageRank

Для оценки важности персонажей используется алгоритм PageRank:

$$PR(v) = \frac{(1 - d)}{N} + d \sum_{u \in B_v} \frac{PR(u)}{L(u)}$$

*Формула 9 — Алгоритм PageRank для оценки важности вершин графа*

Алгоритм позволяет выявлять персонажей, обладающих высокой значимостью в структуре игровых взаимодействий.

#### 1.6.2. Графовые признаки

В рамках работы используются следующие графовые признаки: degree centrality, clustering coefficient, neighborhood metrics, pagerank, graph communities. Использование графовых характеристик позволяет улучшить качество рекомендаций и выявлять скрытые зависимости между персонажами.

### 1.7. Embedding-представления игровых персонажей

Современные рекомендательные системы активно используют embedding-представления объектов. Embedding представляет собой компактное векторное представление объекта в латентном пространстве.

В рамках работы embedding-представления используются для: выявления скрытого сходства персонажей, поиска семантически близких героев, построения latent communities.

Для визуализации embeddings используется Principal Component Analysis (PCA) и t-distributed Stochastic Neighbor Embedding (t-SNE).



Использование embeddings позволяет перейти от ручного описания признаков к автоматическому выявлению скрытых закономерностей в игровом пространстве.

### **1.8. Выводы по главе**

В результате анализа предметной области и существующих подходов было установлено, что задача подбора игровых персонажей в Dota 2 относится к классу сложных контекстно-зависимых recommendation-задач с высокой комбинаторной сложностью и большим количеством скрытых взаимодействий между объектами.

Проведенный обзор показал, что традиционные рекомендательные подходы обладают рядом ограничений и недостаточно эффективно учитывают следующие признаки: синергию персонажей, контекст текущего драфта, графовую структуру взаимодействий, скрытые зависимости между игровыми сущностями.

Наиболее перспективным подходом для решения поставленной задачи является построение гибридной рекомендательной системы, объединяющей методы:

- learning-to-rank
- gradient boosting
- graph analytics
- embedding representations
- feature engineering

Полученные выводы определяют архитектурные и методологические решения, используемые в последующих главах выпускной квалификационной работы, посвященных проектированию, реализации и экспериментальному исследованию разработанной рекомендательной системы.

## **Глава 2. Сбор и анализ данных для рекомендательной системы**

### **2.1. Источники данных и структура игрового датасета**

Для построения интеллектуальной рекомендательной системы подбора игровых персонажей был сформирован специализированный датасет матчей Dota 2, содержащий информацию о составах команд, ролях игроков, результатах матчей и взаимодействиях между персонажами. Основными источниками данных в работе выступили 2 сайта которые хранят и предоставляют статистику по игрокам и матчам: OpenDota API, STRATZ API и официальные игровые данные Dota 2

Сбор данных осуществлялся в несколько последовательных этапов. На первом этапе при помощи OpenDota API производился отбор матчей, удовлетворяющих заданным критериям качества данных. В выборку включались только матчи: актуальных игровых патчей, рейтингового режима, стандартного формата 5×5, с корректно определенными ролями игроков, длительностью более 25 минут.

Подобная фильтрация позволила исключить нерепрезентативные игры, слишком короткие матчи, аномальные игровые сценарии, тренировочные и нестандартные режимы.

После формирования списка идентификаторов матчей выполнялся второй этап обработки данных с использованием STRATZ API. Для каждого матча извлекалась подробная информация о составе команд и игровых характеристиках участников.

В результате была реализована автоматизированная ETL-процедура обеспечивающая извлечение данных их очистку, преобразование, сохранение и построение обучающего датасета.

В процессе реализации системы использовалась многоуровневая структура хранения данных, включающая: исходные игровые данные, промежуточные таблицы признаков, графовые представления, embedding-

представления, parquet-датасеты для ranking-задач и итоговые обучающие выборки.

Подобная архитектура позволила разделить этапы извлечения, предобработки, feature engineering и обучения модели, а также существенно упростить масштабирование системы и повторное использование признаков.

Для каждого игрока внутри матча формировалась отдельная обучающая запись. Таким образом, один матч генерировал 10 строк обучающих данных — по одной для каждого игрового персонажа.

*Таблица 2.1 – Структура исходного матчевого датасета*

| Поле           | Описание          |
|----------------|-------------------|
| target_hero    | Целевой герой     |
| ally_hero_1-4  | Союзные персонажи |
| enemy_hero_1-5 | Противники        |
| label          | Результат матча   |
| lane           | Игровая линия     |
| role           | Игровая роль      |
| patch          | Версия патча      |

Пример строки датасета:

11,29,119,131,86,31,93,12,138,15,1,SAFE\_LANE,CORE,182

где:

11 — идентификатор выбранного героя

значения 29–86 — союзники

значения 31–15 — противники

label = 1 означает победу

SAFE\_LANE и CORE определяют игровую позицию

patch содержит идентификатор версии игрового обновления.

Подобный формат представления данных позволяет интерпретировать задачу как задачу контекстного ранжирования игровых персонажей.

После выполнения процедур фильтрации, очистки и предобработки был сформирован итоговый обучающий датасет следующего объема:

*Таблица 2.2 – Основные характеристики итогового датасета*

| Параметр                     | Значение  |
|------------------------------|-----------|
| Количество записей           | 881 290   |
| Количество признаков         | 14        |
| Количество уникальных героев | 127       |
| Количество игровых патчей    | 1         |
| Размер датасета              | 176.32 MB |

Каждая строка датасета соответствовала одному игровому персонажу в рамках конкретного матча. Благодаря подобному представлению система могла анализировать эффективность героя в контексте текущего состава команды и выбранных противников.

Баланс классов в итоговом датасете оказался полностью симметричным:

*Таблица 2.3 – Распределение целевой переменной*

| Класс                 | Количество | Доля   |
|-----------------------|------------|--------|
| Победа (label = 1)    | 440 645    | 50.0 % |
| Поражение (label = 0) | 440 645    | 50.0 % |

Сбалансированность классов является важным фактором для корректного обучения ranking-модели и позволяет избежать смещения алгоритма в сторону доминирующего класса.

В процессе анализа датасета было исследовано распределение игровых ролей персонажей.

Таблица 2.4 – Распределение игровых ролей

| Роль          | Количество | Доля    |
|---------------|------------|---------|
| CORE          | 529 999    | 60.14 % |
| LIGHT_SUPPORT | 175 774    | 19.95 % |
| HARD_SUPPORT  | 175 517    | 19.92 % |

Полученное распределение соответствует реальной игровой мете Dota 2, в которой основную часть состава команды составляют core-персонажи, ориентированные на нанесение урона и масштабирование в поздней стадии игры.

Также был выполнен анализ распределения персонажей по игровым линиям.

Таблица 2.5 – Распределение игровых линий

| Линия     | Количество | Доля    |
|-----------|------------|---------|
| SAFE_LANE | 351 719    | 39.91 % |
| OFF_LANE  | 349 999    | 39.71 % |
| MID_LANE  | 176 710    | 20.05 % |
| ROAMING   | 1 255      | 0.14 %  |
| JUNGLE    | 951        | 0.11 %  |
| UNKNOWN   | 656        | 0.07 %  |

Наибольшую долю выборки составляют стандартные игровые линии SAFE\_LANE и OFF\_LANE, что соответствует типичной структуре матчей. В рамках анализа датасета была исследована частота выбора героев.

Таблица 2.6 – Наиболее популярные герои датасета

| Герой  | Количество выборов |
|--------|--------------------|
| Pudge  | 247 600            |
| Lion   | 232 890            |
| Rubick | 186 790            |

| Герой          | Количество выборов |
|----------------|--------------------|
| Spirit Breaker | 179 920            |
| Invoker        | 173 280            |
| Sniper         | 169 490            |

Полученные результаты отражают особенности текущей игровой меты и подтверждают репрезентативность собранной выборки относительно реальных матчей Dota 2.

Помимо основного матчевого датасета в процессе feature engineering и подготовки ranking-модели формировались дополнительные таблицы и специализированные наборы данных.

*Таблица 2.7 – Основные датасеты и feature-хранилища системы*

| Датасет                       | Назначение                    |
|-------------------------------|-------------------------------|
| dataset.csv                   | Основной матчевый датасет     |
| hero_synergy_matrix.csv       | Матрица синергий персонажей   |
| hero_counter_matrix.csv       | Матрица контрпигов            |
| hero_graph_features.csv       | Графовые признаки героев      |
| hero_embeddings_v6.csv        | Embedding-представления       |
| hero_stats.csv                | Глобальные статистики героев  |
| hero_roles.csv                | Информация о ролях            |
| hero_similarities.csv         | Сходства embedding-векторов   |
| hard_negative_samples.parquet | Сложные отрицательные примеры |
| training_dataset_hard.parquet | Финальный ranking-датасет     |
| hard_chunks/*.parquet         | Чанки для batch-обработки     |

Для обработки большого объема ranking-данных использовался формат Apache Parquet, оптимизированный для аналитических вычислений и batch-обработки.

В процессе подготовки данных итоговый ranking-датасет разделялся на отдельные parquet-чанки, что позволило снизить потребление оперативной памяти, ускорить feature engineering, повысить масштабируемость pipeline, реализовать потоковую обработку данных.

В результате была сформирована полноценная инфраструктура хранения и обработки признаков, обеспечивающая эффективную подготовку данных для последующего обучения рекомендательной ranking-системы.

## **2.2. Архитектура сбора и обработки данных**

Процесс подготовки данных для рекомендательной системы был реализован в виде многоэтапного pipeline обработки матчевой информации Dota 2, обеспечивающего полный цикл формирования обучающей выборки для ranking-модели.

Разработанная архитектура ориентирована на обработку большого объема игровых данных и включает последовательные этапы извлечения, очистки, преобразования и подготовки признаков для последующего машинного обучения. При проектировании pipeline учитывались: обработка больших объемов матчевых данных, модульность архитектуры, возможность повторного запуска отдельных этапов, масштабируемость feature engineering, поддержка batch-обработки, разделение ETL-процессов и ML-компонентов, минимизация потребления оперативной памяти, возможность обновления данных при выходе новых игровых патчей.

В рамках реализации были разработаны:

- сбор match ID
- загрузка матчей
- очистка данных

- предобработка
- генерация признаков
- формирование графовых структур
- генерация embedding-представлений
- формирование ranking-датасета
- обучение моделей
- inference и генерация рекомендаций
- Этап извлечения данных

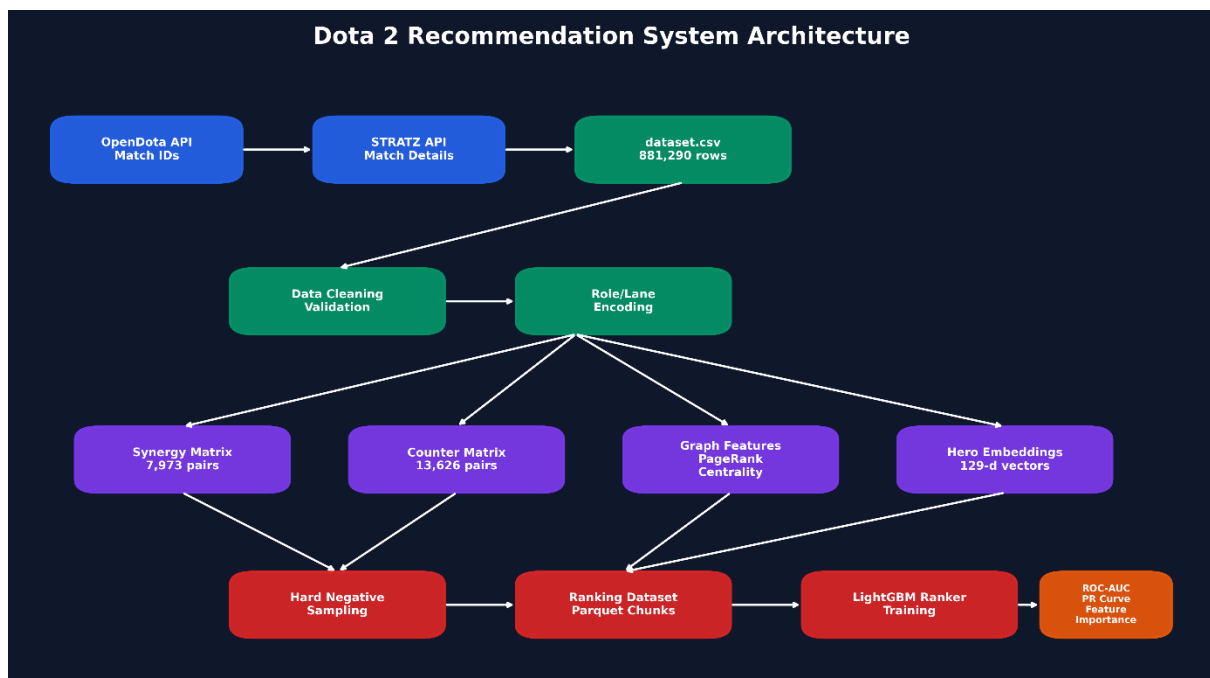


Рисунок 2.3 – Архитектура pipeline обработки данных

На первом этапе выполнялся автоматизированный сбор идентификаторов матчей с использованием OpenDota API. Для обеспечения качества обучающей выборки производилась предварительная фильтрация матчей по следующим критериям: рейтинговый игровой режим, стандартный формат, актуальный игровой патч, корректно определенные роли игроков, минимальная продолжительность матча 25 минут (так как матчи которые заканчиваются быстрее не несут никакой ценности).

После формирования списка match ID запускался второй этап ETL-процесса — загрузка подробной информации о матчах через STRATZ API.



Для каждого матча извлекались: составы команд, идентификаторы героев, игровые роли, игровые линии, результаты матчей, принадлежность к стороне Radiant/Dire, версия игрового патча.

Для обеспечения стабильности работы системы были реализованы: ограничение частоты запросов, механизм повторной отправки запросов, сохранение промежуточного прогресса, обработка ошибок API, восстановление процесса после прерывания.

Подобный подход позволил обеспечить устойчивую обработку большого количества матчей без потери данных и сбоев.

Этап очистки и предобработки данных.

После извлечения матчевой информации выполнялась процедура очистки и предобработки данных, в ходе которых удалялись: неполные матчи, записи с отсутствующими ролями, Поврежденные API-ответы, дубликат, аномальные игровые сценарии, матчи недостаточной продолжительности и т.д.

Далее был сформирован основной матчевый датасет `dataset.csv`, содержащий 881 290 обучающих записей..

После формирования базового датасета запускался этап генерации признаков. Pipeline feature engineering был реализован как отдельный независимый модуль обработки для того, чтобы: повторно использовать вычисленные признаки, ускорить обновление данных, минимизировать повторные обращения к API, упростить масштабирование системы.

На этапе feature engineering были сформированы: статистические признаки, ролевые признаки, признаки игровых линий, матрицы синергий, матрицы контрпиков, графовые признаки, embedding-представления, similarity-признаки, ranking-признаки. Общая структура feature engineering pipeline представлена на рисунке 2.4.

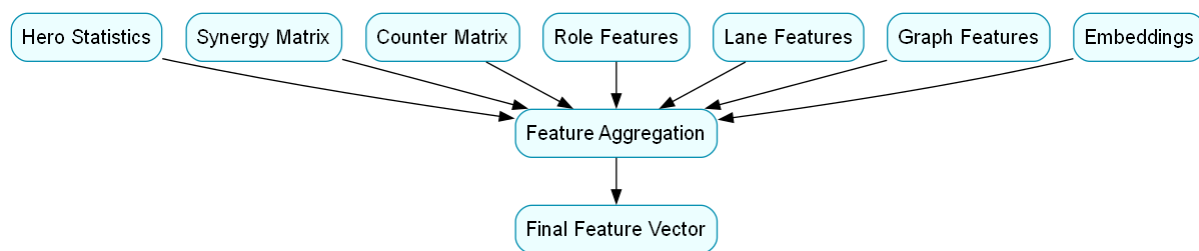


Рисунок 2.4 – Pipeline генерации признаков

Результатом работы pipeline являлось формирование набора промежуточных feature-таблиц:

- hero\_stats.csv
- hero\_synergy\_matrix.csv
- hero\_counter\_matrix.csv
- hero\_graph\_features.csv
- hero\_embeddings\_v6.csv
- hero\_similarities.csv
- hero\_roles.csv

Полученные признаки далее были объединены в единое признаковое пространство для последующего обучения ranking-модели.

После завершения feature engineering выполнялось построение итогового ranking-датасета. Для подготовки обучающей выборки использовался подход hard negative sampling, позволяющий генерировать сложные отрицательные примеры, статистически близкие к успешным игровым комбинациям. В процессе формирования ranking-датасета создавались: положительные примеры, отрицательные примеры, pairwise ranking-пары, hard negative candidates. Итоговые данные сохранялись в формате Apache Parquet:

Дополнительно датасет автоматически разделялся на parquet-чанки внутри директории hard\_chunks/, что позволило существенно снизить потребление оперативной памяти при batch-обработке больших объемов данных.

Заключительным этапом pipeline являлось обучение моделей машинного обучения на основе сформированного признакового пространства. В рамках работы использовались модели:

LightGBM Ranker

LightGBM Binary Classifier

Hard Negative LightGBM Model.

Обученные модели сохранялись в директории models/ и далее использовались системой генерации рекомендаций.

Основной ranking-моделью системы выступал алгоритм LGBMRanker, обеспечивающий ранжирование игровых персонажей по степени релевантности относительно текущего состояния драфта.

Таким образом, разработанная архитектура pipeline обеспечивает полный цикл подготовки данных — от извлечения матчевой информации до формирования итогового ranking-датасета и обучения рекомендательной модели.

### **2.3. Статистические признаки**

Одним из базовых этапов feature engineering являлось построение статистических признаков игровых персонажей. Использование статистических характеристик позволяет модели учитывать текущую эффективность героев, популярность персонажей, особенности игровой меты и ролевую специфику. Статистические признаки формировались на основе обработанного матчевого датасета dataset.csv. Полученные данные сохранялись в таблице hero\_stats.csv.

Использование статистических признаков позволяет учитывать глобальную игровую мету и выявлять героев, демонстрирующих наиболее высокую эффективность в рамках текущего игрового патча.

Также из ключевых статистических признаков являлся глобальный winrate героя.

Показатель вычислялся как отношение количества побед персонажа к общему количеству матчей:

$$WinRate(H) = \frac{Wins(H)}{Matches(H)}$$

*Формула 2.1 — Вычисление процента побед героя*

где:

$Wins(H)$  — количество побед героя

$Matches(H)$  — общее количество матчей героя.

Высокое значение winrate свидетельствует о высокой эффективности персонажа в текущей игровой мете.

Дополнительно вычислялся показатель pickrate, отражающий популярность героя среди игроков. Формула вычисления pickrate имеет вид:

$$PickRate(H) = \frac{Matches(H)}{TotalMatches}$$

*Формула 2.2 — Вычисление частоты выбора героя*

где:

$Matches(H)$  — количество матчей героя

$TotalMatches$  — общее количество матчей в датасете.

Показатель popularity позволяет модели учитывать актуальность героя и распространенность его использования в матчах.

Поскольку эффективность персонажей существенно зависит от выбранной игровой роли, в рамках feature engineering дополнительно вычислялись role-based признаки. Для каждого героя определялись: основная роль, распределение ролей, role winrate, role pickrate, эффективность на различных линиях. Распределение ролей в обучающем датасете представлено следующим образом:

- CORE — 60.14%
- LIGHT\_SUPPORT — 19.95%
- HARD\_SUPPORT — 19.92%

Дополнительно учитывались игровые линии: SAFE\_LANE, OFF\_LANE, MID\_LANE, ROAMING, JUNGLE.

Использование role/lane признаков позволило учитывать контекст использования героя внутри командного драфта. В процессе анализа датасета были выявлены наиболее популярные персонажи текущей игровой меты. Высокая популярность персонажей обусловлена их универсальностью, высокой вариативностью, устойчивостью в различных игровых сценариях, эффективностью в текущем игровом патче.

Дополнительно был выполнен анализ распределения побед и частоты использования героев, что позволило выявлять потенциально переоцененные или недооцененные игровые стратегии.

В результате обработки матчевой информации были сформированы новые таблицы.

*Таблица 2.8 Основные таблицы, сформированные в процессе обработки игровых данных*

| Таблица           | Назначение                   |
|-------------------|------------------------------|
| hero_stats.csv    | Глобальная статистика героев |
| hero_roles.csv    | Ролевые характеристики       |
| dataset.csv       | Основной матчевый датасет    |
| hero_registry.csv | Справочник героев            |

#### 2.3.1. Матрица синергий

Одной из ключевых особенностей многопользовательских МОБА-игр является наличие сложных взаимодействий между игровыми персонажами. Эффективность героя определяется не только его индивидуальными характеристиками, но и совместимостью с союзными персонажами внутри командного драфта.

В связи с этим одним из наиболее важных этапов feature engineering являлось построение матрицы синергий героев. Под синергией в рамках данной работы понимается статистически значимое повышение вероятности победы при совместном использовании двух персонажей в одной команде. Использование признаков синергии позволяет рекомендательной системе учитывать: совместимость способностей

героев, эффективность командных комбинаций, синергию контроля, взаимодействие ультимативных способностей, усиление игровых стратегий, совместимость ролей внутри команды.

Подобный подход позволяет перейти от анализа отдельных персонажей к анализу полноценных игровых взаимодействий.

Для каждого pairwise-сочетания союзных героев вычислялся показатель совместной эффективности относительно глобального среднего winrate. Формально показатель синергии определялся следующим образом:

$$Synergy(A, B) = WinRate(A, B) - GlobalWinRate$$

*Формула 2.3 - Вычисление показателя синергии героев*

где:

$WinRate(A, B)$  — вероятность победы при совместном использовании героев  $A$  и  $B$

$GlobalWinRate$  — средний winrate по всему датасету.

Положительное значение показателя синергии означает, что совместное использование героев демонстрирует эффективность выше среднего уровня по выборке. Отрицательное значение, напротив, свидетельствует о низкой совместимости персонажей внутри команды.

В процессе обработки матчевых данных выполнялся анализ всех союзных pairwise-комбинаций героев внутри команды. Для каждой пары вычислялись: количество совместных матчей, количество побед, совместный winrate, отклонение от глобального среднего, статистическая устойчивость комбинации.

Дополнительно производилась фильтрация редких сочетаний персонажей для снижения влияния шумовых взаимодействий и недостаточно репрезентативных игровых комбинаций. Полученные значения сохранялись в таблице:

hero\_synergy\_matrix.csv.

Итоговая таблица содержит 7 973 pairwise-взаимодействия героев, а так же статистики совместных побед, показатели synergy score и вспомогательные признаки взаимодействий.

Использование отдельной таблицы синергий позволило существенно ускорить генерацию признаков при обучении ranking-модели и inference-этапе системы рекомендаций.

Для анализа структуры взаимодействий между героями матрица синергий дополнительно интерпретировалась как графовая структура. В рамках графового представления: вершины графа соответствуют игровым персонажам, ребра отражают статистически значимые synergy-взаимодействия, вес ребра определяется величиной synergy score. Подобное представление позволяет анализировать скрытые взаимосвязи между героями и выявлять устойчивые игровые сообщества.

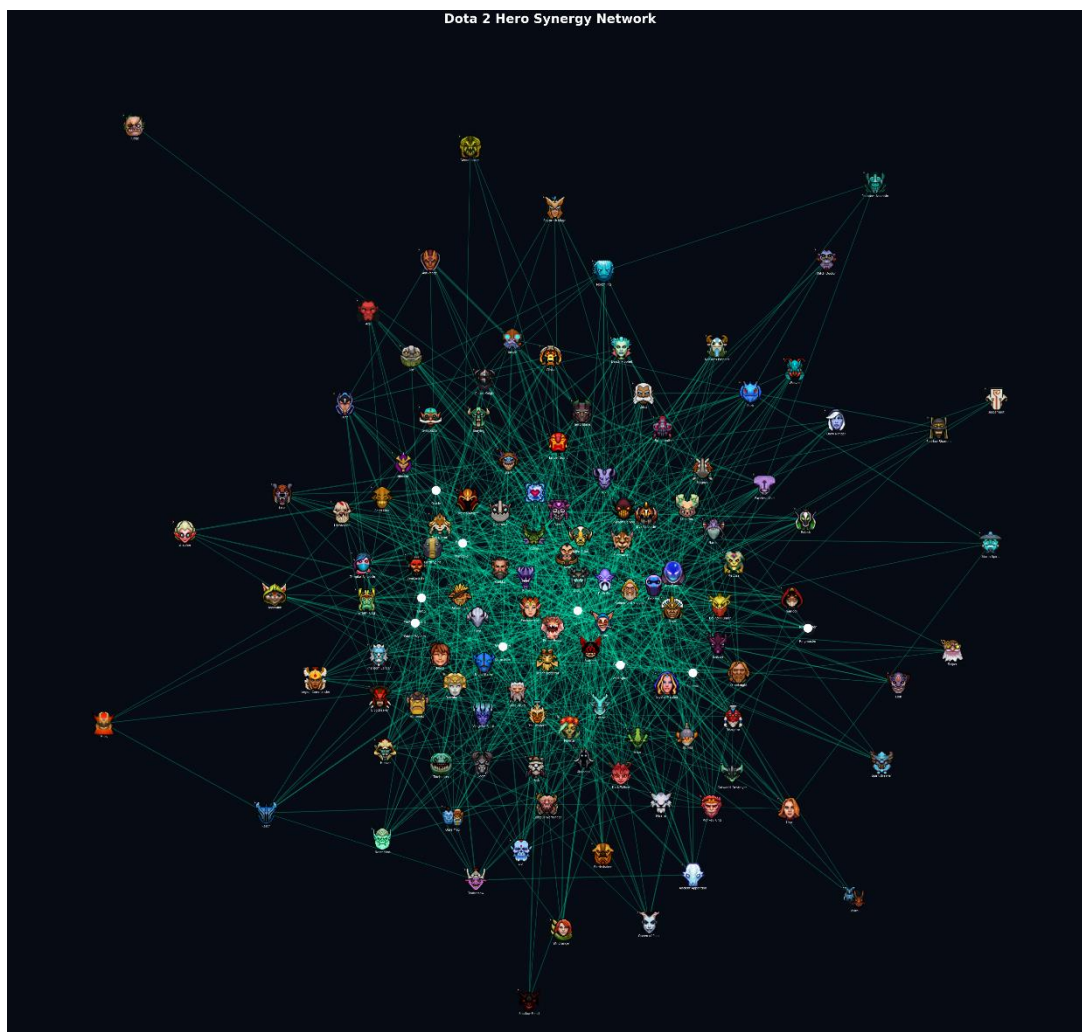


Рисунок 2.5 – Граф синергий игровых персонажей Dota 2

Анализ структуры графа показал наличие устойчивых кластеров героев, формирующих эффективные игровые комбинации. Наиболее выраженные сообщества наблюдаются среди:

- teamfight-oriented персонажей
- initiator-героев
- support-комбинаций
- push-стратегий
- burst-damage связок.

Дополнительно были выявлены герои с высокой степенью graph connectivity, обладающие большим количеством эффективных взаимодействий с другими персонажами. Подобные герои демонстрируют высокую универсальность и способны эффективно интегрироваться в различные командные композиции.

Полученные synergy-признаки использовались в качестве входных признаков для LightGBM Ranker. Для каждого candidate hero вычислялись: средняя синергия с союзниками, максимальная synergy score, суммарная синергия команды, normalized synergy features, graph-based synergy metrics.

Использование признаков синергии позволило модели учитывать контекст текущего драфта и существенно повысило качество итогового ранжирования персонажей. Таким образом, построение матрицы синергий обеспечило рекомендательную систему информацией о скрытых взаимодействиях между героями и позволило учитывать командную совместимость персонажей при генерации рекомендаций.

### 2.3.2. Матрица контрпигов

Помимо анализа союзных взаимодействий между персонажами, важной составляющей рекомендательной системы являлось моделирование противостояний между героями противоположных команд. В многопользовательских МОБА-играх эффективность персонажа существенно зависит от текущего набора противников. Некоторые герои



способны значительно снижать эффективность определенных игровых стратегий, блокировать ключевые способности противника или создавать неблагоприятные игровые сценарии для отдельных персонажей. В связи с этим в рамках работы была построена специализированная матрица контрпигов, отражающая эффективность героя против конкретных противников. Использование counter-признаков позволяет рекомендательной системе учитывать: контрпики персонажей, подавление игровых стратегий, эффективность против определенных ролей, взаимодействие способностей, anti-carry механики, ограничения мобильности и устойчивость к контролю. Подобный подход позволяет системе формировать рекомендации с учетом не только союзных взаимодействий, но и текущего состава противников.

Для оценки эффективности персонажа против конкретного противника вычислялось изменение вероятности победы героя относительно его глобального среднего winrate. Формально показатель контрпики определялся следующим образом:

$$\text{Counter}(A, B) = \text{WinRate}(A \text{ against } B) - \text{GlobalWinRate}(A)$$

*Формула 2.4 — Оценка эффективности героя против противника*

где:

$\text{WinRate}(A \text{ against } B)$  — вероятность победы героя  $A$  против героя  $B$ ;

$\text{GlobalWinRate}(A)$  — глобальный winrate героя  $A$ .

Положительное значение показателя свидетельствует о наличии статистического преимущества героя против конкретного противника. Отрицательное значение, напротив, указывает на неблагоприятный matchup.

В процессе обработки матчевого датасета анализировались все pairwise-взаимодействия между героями противоположных команд. Для каждого matchup вычислялись: количество матчей, количество побед,

winrate против конкретного героя, counter score и статистическая устойчивость взаимодействия.

Для снижения влияния шумовых данных дополнительно выполнялась фильтрация недостаточно репрезентативных matchup-комбинаций. Полученные значения сохранялись в таблице hero\_counter\_matrix.csv. Итоговая таблица содержит 13 626 взаимодействий между героями, показатели counter score, статистики matchup-взаимодействий и дополнительные признаки противостояния персонажей.

Использование предварительно вычисленной матрицы контрпигов позволило существенно ускорить этап генерации признаков и inference ranking-модели.

Для анализа структуры противостояний между героями матрица контрпигов дополнительно интерпретировалась как графовая структура. В рамках данного представления: вершины графа соответствуют игровым персонажам, ребра отражают статистически значимые counter-взаимодействия, вес ребра определяется величиной counter score. Визуализация графа контрпигов представлена на рисунке 2.6.

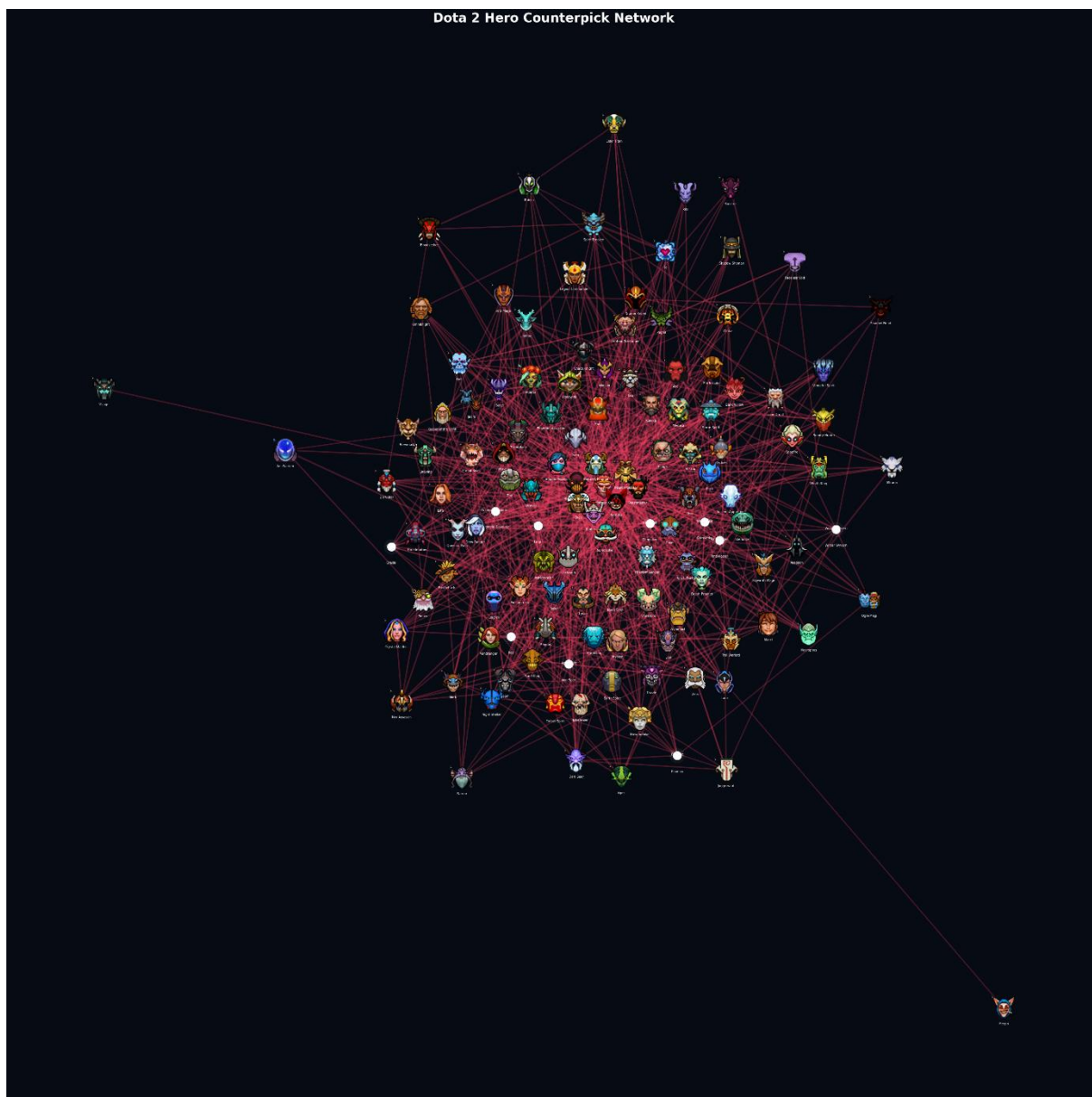


Рисунок 2.6 – Граф контрпигов игровых персонажей Dota 2

Анализ структуры графа позволил выявить: героев с высоким количеством favorable matchups, персонажей, эффективно подавляющих мобильные стратегии, anti-carry комбинации, устойчивые defensive-community структуры и героев с высоким уровнем counter-connectivity.

Дополнительно были выявлены персонажи, обладающие высокой степенью универсальности против различных игровых стратегий.

Подобные герои обладают повышенной ценностью в процессе драфта, поскольку способны эффективно адаптироваться к широкому спектру игровых сценариев.

Полученные признаки контрпикиков использовались в качестве входных признаков для модели ранжирования. Для каждого candidate hero вычислялись: средний counter score против команды противника, максимальное преимущество matchup, количество favorable matchups, normalized counter features, graph-based counter metrics.

Использование counter-признаков позволило существенно повысить качество рекомендаций в условиях динамического драфта и улучшить способность модели учитывать текущий состав противников. Таким образом, построение матрицы контрпикиков обеспечило рекомендательную систему информацией о статистических преимуществах и недостатках персонажей относительно конкретных игровых matchup-взаимодействий.

#### 2.3.4. Графовые признаки

Одним из наиболее важных этапов feature engineering являлось построение графовых признаков, позволяющих учитывать сложные структурные взаимодействия между игровыми персонажами.

В классических статистических подходах взаимодействия между героями рассматриваются как независимые pairwise-комбинации. Однако подобное представление не позволяет учитывать глобальную структуру игровых взаимодействий и скрытые взаимосвязи между персонажами.

Для решения данной проблемы в рамках работы использовались методы graph analytics, позволяющие моделировать пространство игровых взаимодействий в виде графовой структуры. В рамках построенного графа:

На основе матриц синергий и контрпикиков формировался weighted graph взаимодействий между героями. В результате был сформирован граф взаимодействий, содержащий: 129 игровых персонажей, тысячи pairwise-взаимодействий, weighted connectivity между героями.

Полученная графовая структура использовалась для последующего вычисления graph-based признаков.

Одним из базовых графовых признаков являлась степень центральности вершины (degree centrality). Показатель отражает

количество связей героя с другими персонажами внутри графа взаимодействий. Формально degree centrality вычисляется следующим образом:

$$DC(v) = \frac{deg(v)}{(N - 1)}$$

*Формула 2.5 — Расчет центральности вершины по степени связности*

где:

$deg(v)$  — количество связей вершины;

$N$  — количество вершин графа.

Высокое значение centrality свидетельствует о высокой универсальности героя и его способности эффективно взаимодействовать с большим количеством персонажей.

Для оценки значимости персонажей внутри графовой структуры использовался алгоритм PageRank. Алгоритм позволяет учитывать не только количество связей героя, но и важность соседних вершин графа. Использование PageRank позволило выявлять героев, обладающих высокой значимостью внутри структуры игровых взаимодействий. Подобные персонажи демонстрируют устойчивую эффективность в различных игровых сценариях и обладают высокой ценностью в процессе драфта.

Дополнительно вычислялся clustering coefficient, отражающий плотность локальных взаимодействий между героями. Показатель позволяет выявлять: устойчивые игровые сообщества, группы совместимых персонажей, специализированные teamfight-комбинации, локальные synergy-кластеры.

Высокое значение clustering coefficient свидетельствует о наличии плотных игровых связей между группой героев.

Для выявления скрытых игровых сообществ использовались алгоритмы community detection. Подобные методы позволяют автоматически находить группы персонажей, обладающих: схожими

игровыми ролями, похожими стратегиями, высокой совместимостью, близкими graph-based характеристиками.

Выявленные сообщества использовались в качестве дополнительных признаков рекомендательной системы. Полученные признаки сохранялись в таблице `hero_graph_features.csv`. Использование graph-based признаков позволило существенно расширить информационное пространство модели и повысить качество ranking-рекомендаций.

Графовые признаки использовались в качестве дополнительных входных параметров модели LightGBM Ranker. Для каждого candidate hero модель учитывала: позицию героя внутри графа, степень graph-connectivity, принадлежность к игровым сообществам, структурную значимость персонажа, устойчивость взаимодействий.

Подобный подход позволил модели учитывать не только локальные pairwise-взаимодействия, но и глобальную структуру игрового пространства. Использование graph analytics обеспечило рекомендательную систему дополнительной информацией о скрытых зависимостях между героями и позволило существенно повысить качество рекомендаций в условиях сложного контекстного драфта.

#### 2.3.5. Embedding-представления персонажей

Одним из современных подходов к построению рекомендательных систем является использование embedding-представлений объектов в латентном признаковом пространстве.

В отличие от классических статистических признаков, embedding-векторы позволяют моделировать скрытые зависимости между объектами и автоматически выявлять семантическое сходство между игровыми персонажами.

В рамках данной работы embedding-представления использовались для: поиска похожих героев, выявления скрытых игровых зависимостей, построения similarity-признаков, feature augmentation, улучшения качества ranking-рекомендаций.

Каждому игровому персонажу сопоставлялся многомерный embedding-вектор, отражающий его положение в латентном пространстве игровых взаимодействий. Embedding-представления формировались на основе: статистики матчей, synergy-взаимодействий, counter-взаимодействий, graph-based признаков, similarity-взаимодействий и role/lane характеристик. В результате обработки данных был сформирован embedding-датасет hero\_embeddings\_v6.csv. Размерность embedding-пространства составила 129 измерений. Каждая строка embedding-таблицы содержит идентификатор героя, embedding-вектор и дополнительные similarity-признаки.

Использование embedding-представлений позволило перейти от ручного описания игровых характеристик к автоматическому выявлению скрытых закономерностей внутри игрового пространства.

Одним из основных преимуществ embedding-подхода является возможность вычисления similarity между героями в латентном пространстве. Сходство между персонажами вычислялось на основе косинусной меры близости:

$$\text{sim}(x, y) = \frac{(x \cdot y)}{\|x\| \|y\|}$$

*Формула 2.6 — Косинусная мера сходства между векторами*

где:

$x$  и  $y$  — embedding-векторы персонажей.

Высокое значение similarity свидетельствует о близости героев по игровым характеристикам, стилю игры и взаимодействию с другими персонажами. Подобные признаки позволяют рекомендательной системе находить альтернативных героев, выявлять схожие игровые стратегии, а так же учитывать скрытые зависимости между персонажами и улучшать качество рекомендаций при ограниченном контексте драфта.

Для анализа структуры embedding-пространства использовались методы снижения размерности:

Principal Component Analysis (PCA)

t-distributed Stochastic Neighbor Embedding (t-SNE)

Использование PCA позволило визуализировать глобальную структуру embedding-пространства и выявить основные направления распределения игровых персонажей.

Визуализация embedding-пространства методом PCA представлена на рисунке 2.7.

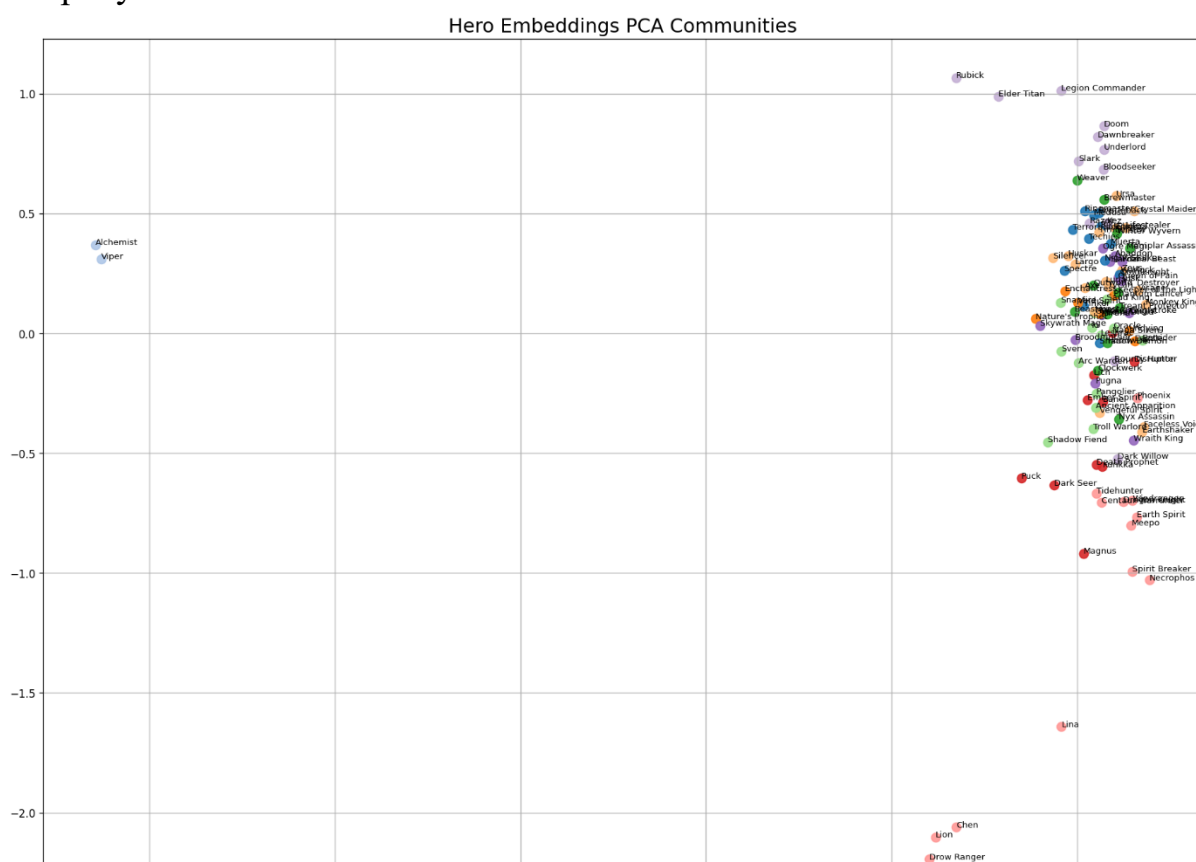


Рисунок 2.7 – PCA-визуализация *embedding*-представлений игровых персонажей

Для более детального анализа локальных игровых сообществ использовался алгоритм t-SNE, позволяющий сохранять локальную структуру расстояний между объектами.



Визуализация embedding-пространства методом t-SNE представлена на рисунке 2.8.

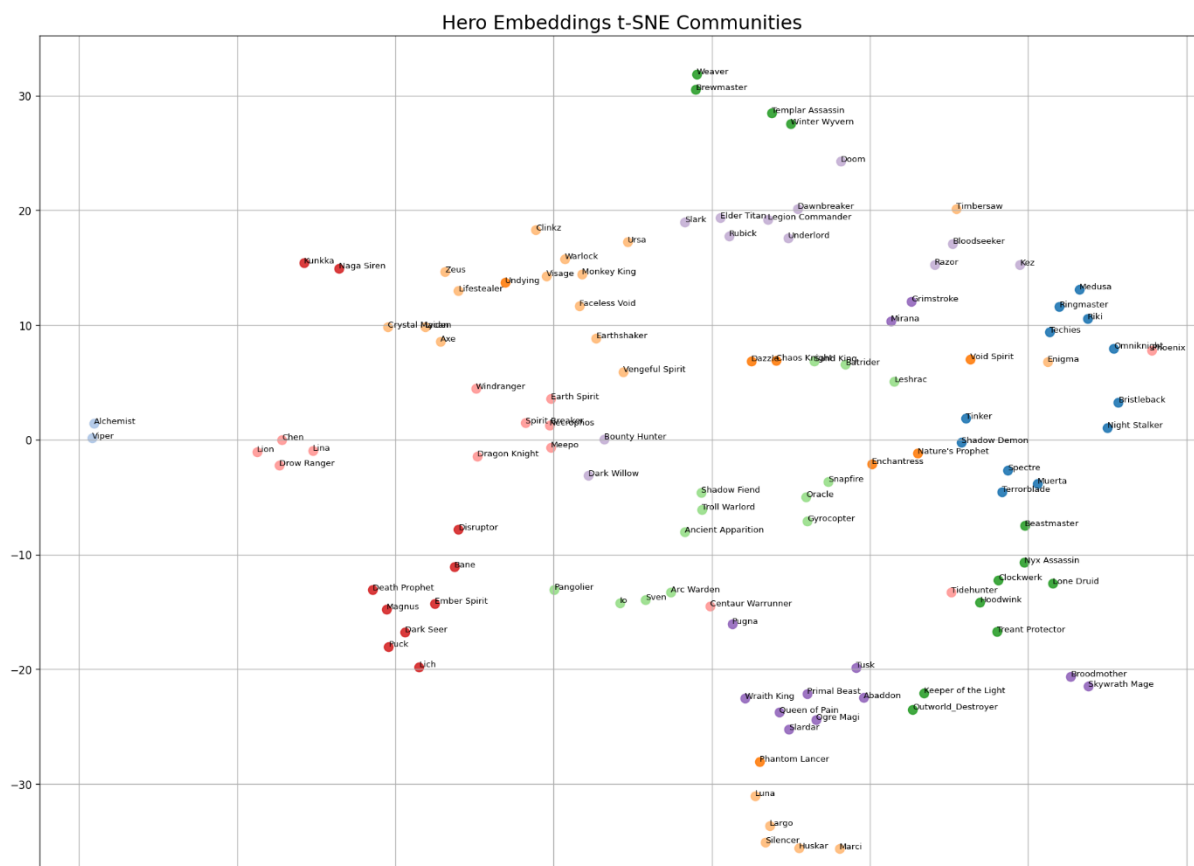


Рисунок 2.8 – t-SNE-визуализация embedding-пространства героев

Анализ embedding-пространства показал наличие выраженных кластеров персонажей, соответствующих:

- игровым ролям
- типам урона
- стилям командного взаимодействия
- мобильным героям
- initiator-персонажам
- support-героям
- carry-кластерам

Подобные результаты подтверждают способность embedding-подхода автоматически выявлять скрытые закономерности внутри игрового пространства.

На основе embedding-векторов дополнительно реализовывался механизм поиска похожих персонажей. Для каждого героя определялись ближайшие соседи в embedding-пространстве. Результаты similarity retrieval сохранялись в таблице hero\_similarities.csv. Подобный подход позволяет: рекомендовать альтернативных героев, учитывать предпочтения пользователя, находить персонажей со схожим стилем игры, реализовывать content-based рекомендации.

Embedding-векторы использовались в качестве дополнительных входных признаков модели LightGBM Ranker. Для каждого candidate hero модель учитывал embedding similarity, расстояния между персонажами, latent interaction features, принадлежность к embedding-community, similarity-связи между героями.

Использование embedding-признаков позволило повысить качество ranking-рекомендаций и улучшить способность модели учитывать скрытые взаимодействия между игровыми персонажами. Таким образом, embedding-представления обеспечили рекомендательную систему механизмом моделирования латентного игрового пространства и позволили учитывать скрытые зависимости между героями, недоступные классическим статистическим методам анализа.

#### 2.3.6. Hard Negative Sampling

Одной из ключевых проблем обучения рекомендательных ranking-систем является низкая информативность случайных отрицательных примеров. При случайной генерации negative samples модель зачастую обучается различать слишком очевидные игровые сценарии, что снижает качество итогового ранжирования. В задаче подбора игровых персонажей данная проблема особенно актуальна, поскольку многие герои обладают схожими статистическими характеристиками, близкими игровыми ролями и сопоставимыми показателями эффективности. В подобных условиях случайные отрицательные примеры не позволяют модели корректно обучаться различению тонких различий между потенциально

подходящими персонажами. Для решения данной проблемы в работе использовался подход *hard negative sampling*, основанный на генерации сложных отрицательных примеров. Под *hard negative examples* понимаются отрицательные объекты, обладающие высокой статистической близостью к положительным примерам, но приводящие к менее эффективным игровым комбинациям. В отличие от случайных отрицательных примеров, *hard negative candidates* характеризуются:

1. схожими *embedding*-представлениями
2. близкими значениями *global winrate*
3. одинаковыми *role/lane* характеристиками
4. сопоставимыми *graph-based* признаками
5. высокой *similarity* близостью
6. близкими показателями популярности персонажей

Использование *hard negative sampling* позволяет существенно усложнить обучающую задачу и сформировать более качественное *ranking-пространство*.

В процессе генерации обучающей выборки для каждого положительного примера дополнительно формировался набор *hard negative candidates*. В качестве *hard negative* объектов выбирались герои: со схожими *embedding*-векторами, с близкими *graph-based* признаками, с аналогичной игровой ролью, с похожими *lane*-характеристиками, с сопоставимым уровнем популярности, с близкими значениями *synergy* и *counter score*. После генерации кандидатов выполнялась дополнительная фильтрация, направленная на исключение:

- полностью случайных героев
- нерелевантных *role/lane* комбинаций
- статистически невалидных сценариев
- несовместимых *draft*-комбинаций

Сформированные hard negative примеры сохранялись в таблице `hard_negative_samples.parquet`.

Использование сложных отрицательных примеров позволило повысить устойчивость ranking-модели к пограничным игровым сценариям и улучшить качество различения близких по характеристикам персонажей.

Для подготовки обучающей выборки использовался pairwise learning-to-rank подход, при котором модель обучалась сравнивать пары игровых персонажей относительно текущего состояния драфта. В рамках данного подхода формировались: положительные примеры успешных игровых комбинаций, отрицательные примеры менее эффективных решений, hard negative pairs, ranking-кандидаты для последующего ранжирования.

Для каждой пары объектов модель обучалась определять, какой из героев является более предпочтительным в контексте текущего состава команд. Функция ranking loss минимизировала различие между score положительных и отрицательных объектов:

$$L = -\sum(i, j) \log \sigma(s_i - s_j)$$

*Формула 2.7 — Функция потерь для ranking-модели*

где:

$s_i$  — score положительного объекта;

$s_j$  — score отрицательного объекта.

Подобный подход позволяет модели обучаться корректному относительному ранжированию персонажей вместо бинарной классификации.

После завершения процедуры hard negative sampling формировался итоговый ranking-датасет под названием `training_dataset_hard.parquet`.

Для повышения эффективности обработки больших объемов данных итоговый датасет автоматически разделялся на parquet-чанки внутри директории `hard_chunks/`. Использование chunk-based архитектуры позволило: снизить потребление оперативной памяти, ускорить batch-обработку, повысить масштабируемость pipeline и оптимизировать

подготовку данных для ranking-моделей. Итоговый ranking-датасет включал:

- статистические признаки
- graph-based признаки
- embedding-представления
- synergy features
- counter features
- role/lane признаки
- pairwise ranking-пары

Таким образом, использование hard negative sampling позволило сформировать более сложное и информативное ranking-пространство, обеспечивающее подготовку качественной обучающей выборки для последующего обучения рекомендательной модели.

#### 2.3.7. Формирование итогового признакового пространства

После завершения этапов feature engineering выполнялось объединение всех вычисленных признаков в единое признаковое пространство, используемое для последующего обучения ranking-моделей. Основной задачей данного этапа являлось формирование комплексного feature vector, способного одновременно учитывать: статистические характеристики героев, синергию персонажей, контрпики, graph-based зависимости, embedding-представления, role/lane особенности, similarity-признаки, contextual draft information.

В процессе объединения признаков использовались данные из таблиц:

1. dataset.csv
2. hero\_synergy\_matrix.csv
3. hero\_counter\_matrix.csv
4. hero\_graph\_features.csv
5. hero\_embeddings\_v6.csv

6. hero\_stats.csv

7. hero\_similarities.csv

Каждая запись итогового ranking-датасета содержала: идентификатор целевого героя, союзных персонажей, противников, role/lane признаки, embedding-вектор, graph-based признаки, synergy features, counter features, статистические характеристики героя, признаки similarity.

Формирование итогового feature vector выполнялось в несколько последовательных этапов: Загрузка базового матчевого датасета, Объединение статистических признаков, Добавление synergy и counter features, Интеграция graph-based признаков, Добавление embedding-представлений, Формирование role/lane encoding, Генерация ranking labels, Добавление hard negative examples. После объединения всех источников данных формировалось итоговое признаковое пространство высокой размерности.

Поскольку итоговый ranking-датасет содержал большой объем обучающих записей, для хранения использовался формат Apache Parquet.

Дополнительно датасет автоматически разделялся на parquet-чанки внутри директории hard\_chunks/.

Подобная архитектура позволяла обрабатывать large-scale ranking datasets без необходимости полной загрузки данных в оперативную память.

Сформированное признаковое пространство использовалось в качестве входных данных для последующего обучения ranking-моделей рекомендательной системы.

Таким образом, в результате выполнения этапов feature engineering был построен комплексный ranking-датасет, объединяющий статистические, графовые и embedding-признаки, что позволило подготовить информативное пространство признаков для решения задачи интеллектуального подбора игровых персонажей Dota 2.

## Выводы по главе 2

Во второй главе была рассмотрена процедура сбора, обработки и подготовки игровых данных для построения рекомендательной системы подбора персонажей Dota 2.

В рамках работы были исследованы основные источники игровых данных, включая OpenDota API и STRATZ API, на основе которых был сформирован специализированный матчевый датасет, содержащий информацию о составах команд, ролях игроков, игровых линиях, результатах матчей и взаимодействиях между персонажами.

В результате реализации ETL-процесса был сформирован основной обучающий датасет `dataset.csv`, содержащий 881 290 записей и охватывающий 129 игровых персонажей. Дополнительно были реализованы процедуры очистки, фильтрации и валидации матчевой информации, позволившие исключить неполные и нерепрезентативные игровые сценарии.

- В ходе feature engineering были построены:
- статистические признаки героев
- матрицы синергий
- матрицы контрпики
- graph-based признаки
- embedding-представления персонажей
- role/lane признаки
- признаки центральности графа
- similarity-признаки

Для анализа взаимодействий между персонажами были сформированы графовые структуры, отражающие синергии и контрпики героев. На основе графов дополнительно вычислялись показатели PageRank, degree centrality, clustering coefficient и community labels, позволяющие учитывать сложные нелинейные зависимости между игровыми персонажами.

Дополнительно был реализован механизм *hard negative sampling*, обеспечивающий генерацию сложных отрицательных примеров и формирование *pairwise ranking*-пар для последующего обучения *ranking*-моделей.

Результатом выполнения всех этапов *preprocessing* и *feature engineering* стало построение комплексного признакового пространства, объединяющего статистические, графовые и *embedding*-признаки, а также формирование итогового *ranking*-датасета в формате *Apache Parquet*.

Полученный датасет и сформированное признаковое пространство используются в следующей главе для обучения, тестирования и оценки качества рекомендательной *ranking*-системы подбора персонажей *Dota 2*.



## Глава 3. Разработка и тестирование рекомендательной системы

### 3.1 Архитектура рекомендательной системы

После завершения этапов сбора данных и формирования признакового пространства была реализована интеллектуальная рекомендательная система подбора игровых персонажей Dota 2, предназначенная для генерации персонализированных рекомендаций в условиях динамического игрового драфта.

Основной задачей системы являлось ранжирование игровых персонажей по степени релевантности относительно текущего состояния матча, состава союзной команды и выбранных противников.

Разработанная система реализует гибридный recommendation-подход, объединяющий: statistical features, graph analytics, embedding representations, contextual recommendation, learning-to-rank, gradient boosting. В качестве основной ranking-модели использовался алгоритм LightGBM Ranker, обеспечивающий эффективное ранжирование игровых персонажей на основе сложного многомерного признакового пространства.

Архитектура системы была спроектирована с учетом следующих требований:

- поддержка real-time рекомендаций
- масштабируемость inference pipeline
- модульность компонентов
- возможность повторного обучения моделей
- поддержка feature engineering
- низкое время генерации рекомендаций

В состав рекомендательной системы входят следующие основные компоненты: модуль загрузки данных, модуль feature engineering, ranking-

модель, inference pipeline, recommendation engine, пользовательский интерфейс, система визуализации результатов.

Входными данными системы являются: текущий состав союзной команды, состав команды противника, выбранные игровые роли, lane-позиции, contextual draft information.

На основе входных параметров система формирует feature vector и вычисляет ranking score для каждого кандидата.

После вычисления ranking score выполняется сортировка героев по степени релевантности, в результате чего пользователь получает список наиболее подходящих персонажей для текущего игрового сценария.

Ключевым компонентом архитектуры является inference pipeline, обеспечивающий последовательную обработку текущего игрового драфта и генерацию рекомендаций.

Входными данными системы выступает текущий состав команд (Current Draft), содержащий информацию о выбранных союзных и вражеских персонажах. На основе текущего драфта выполняется генерация признаков (Feature Generator), включающая:

- статистические признаки героев
- role/lane признаки
- synergy features
- counter features
- contextual draft features

Далее система вычисляет graph-based признаки (Graph Features). После этого формируются embedding-признаки (Embedding Features), описывающие скрытые similarity-зависимости между персонажами в латентном признаковом пространстве. Полученное признаковое представление передается в ranking-модель LightGBM Ranker, выполняющую вычисление score релевантности для каждого кандидата.

На следующем этапе производится оценка кандидатов (Candidate Scoring), в рамках которой для каждого героя вычисляется итоговый ranking score относительно текущего игрового контекста.

После вычисления ranking score выполняется этап Top-K Ranking, в ходе которого кандидаты сортируются по степени релевантности.

Результатом работы системы является формирование итоговых рекомендаций (Recommendations), представляющих собой список наиболее подходящих персонажей для текущего игрового драфта.

Inference pipeline рекомендательной системы представлен на рисунке

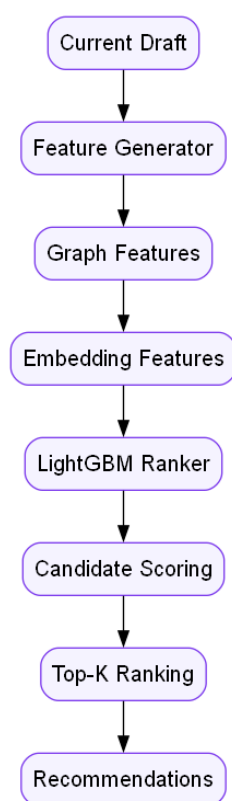


Рисунок 3.1 – Inference pipeline рекомендательной системы

### 3.2 Обучение ranking-модели на основе LightGBM

После формирования итогового признакового пространства выполнялось обучение ranking-модели, предназначенной для ранжирования игровых персонажей в контексте текущего драфта.

В качестве основной модели в работе использовался алгоритм LGBMRanker библиотеки LightGBM, реализующий подход Learning-to-Rank на основе градиентного бустинга над решающими деревьями.

Выбор LightGBM Ranker был обусловлен рядом преимуществ:

- высокой скоростью обучения
- эффективной обработкой больших датасетов
- поддержкой ranking-задач
- устойчивостью к разнородным признакам
- возможностью работы с большим количеством feature-признаков
- высокой качественной интерпретируемостью результатов

В отличие от классической бинарной классификации, ranking-подход позволяет модели обучаться относительному ранжированию кандидатов внутри одного игрового контекста, а не только прогнозированию вероятности победы отдельного героя.

Модель обучалась назначать более высокий ranking score корректным игровым решениям по сравнению с менее эффективными вариантами выбора героя. Подобный подход позволил приблизить процесс обучения к реальному сценарию работы рекомендательной системы.

В процессе обучения модель формировала ranking score для каждого героя-кандидата относительно текущего игрового драфта.

Основной задачей модели являлось ранжирование персонажей по степени их релевантности в конкретной игровой ситуации.

Обученная модель сохранялась в директории `models/lgbm_ranker.pkl`.

Дополнительно в рамках работы были обучены:

- LightGBM Binary Classifier
- Hard Negative LightGBM Model

После завершения обучения выполнялся анализ feature importance модели. Наибольший вклад в итоговое качество ранжирования внесли: candidate hero, avg synergy, avg counter, max synergy, worst counter, role encoded, lane encoded

Дополнительно значимыми оказались: PageRank, clustering coefficient, degree centrality, embedding similarity

Полученные результаты подтверждают высокую значимость как статистических, так и graph-based признаков в задаче рекомендаций игровых персонажей. Результаты анализа feature importance представлены на рисунке 3.2.

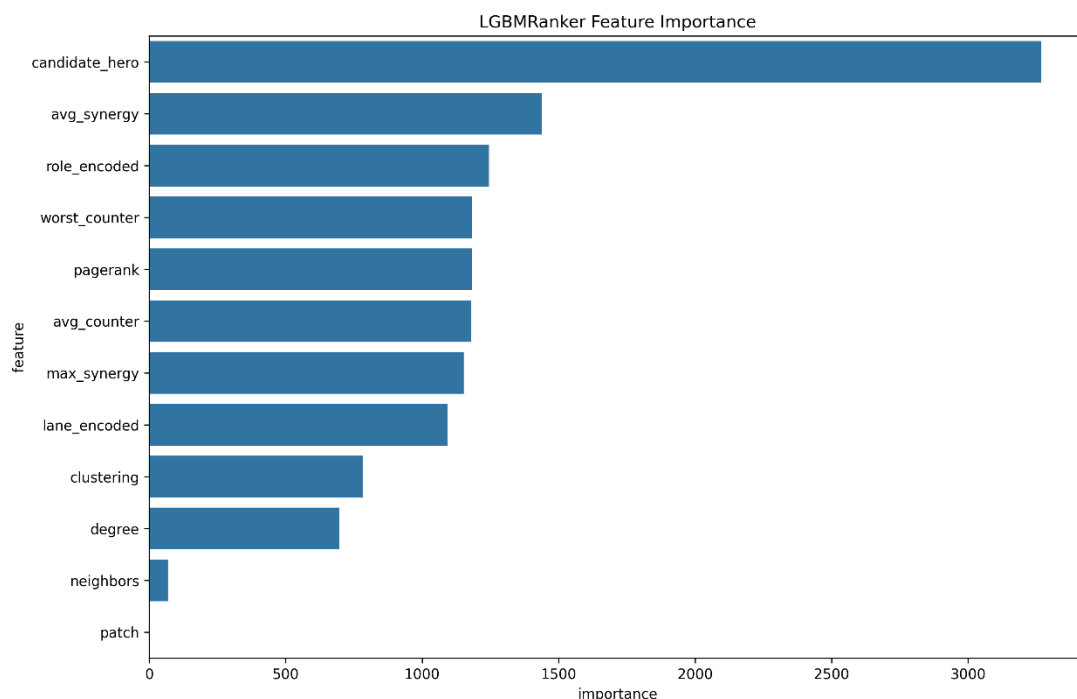


Рисунок 3.2 – Важность признаков модели LGBMRanker

Анализ feature importance показал, что наибольший вклад в итоговое ранжирование игровых персонажей вносит признак `candidate_hero`, отражающий идентификатор рассматриваемого героя-кандидата. Высокая значимость данного признака объясняется наличием устойчивых игровых паттернов и различий в эффективности персонажей внутри текущей игровой меты. Полученные результаты подтверждают, что модель активно использует информацию о взаимодействиях между персонажами при формировании рекомендаций. Особенно важным фактором является совместимость героя с союзниками и эффективность против текущего состава противников. Дополнительно существенный вклад внесли `role_encoded` и `lane_encoded`, что свидетельствует о высокой зависимости

качества рекомендаций от игровой позиции персонажа и распределения ролей внутри команды.

Высокая значимость PageRank подтверждает эффективность использования графовых методов анализа игровых взаимодействий. Модель учитывает не только локальные статистические показатели героя, но и его структурную роль внутри глобального графа синергий и контрпигов.

Наличие значимых embedding и graph-based признаков демонстрирует, что использование комбинированного feature space позволяет учитывать скрытые нелинейные зависимости между персонажами, которые невозможно выявить при использовании исключительно статистических методов.

Таким образом, результаты анализа feature importance подтверждают эффективность предложенного подхода к построению гибридного признакового пространства для ranking-рекомендательной системы Dota 2.

Использование Learning-to-Rank подхода позволило: учитывать относительную релевантность героев, корректно ранжировать кандидатов, улучшить качество Top-K рекомендаций, повысить устойчивость модели к сложным игровым сценариям, снизить влияние случайных игровых комбинаций.

Дополнительным преимуществом являлась высокая скорость inference, что особенно важно для интерактивной рекомендательной системы реального времени.

В результате обученная модель использовалась в качестве основного ядра рекомендательной системы и обеспечивала формирование персонализированных рекомендаций игровых персонажей для текущего драфта. Дополнительным преимуществом являлась высокая скорость inference, что особенно важно для интерактивной рекомендательной системы реального времени.

### 3.4. Оценка качества ranking-модели

После завершения обучения выполнялась комплексная оценка качества разработанной рекомендательной модели.

Для оценки качества использовались следующие метрики:

- ROC-AUC
- PR-AUC
- confusion matrix
- feature importance analysis

Одной из основных метрик оценки выступала ROC-AUC (Receiver Operating Characteristic Area Under Curve).

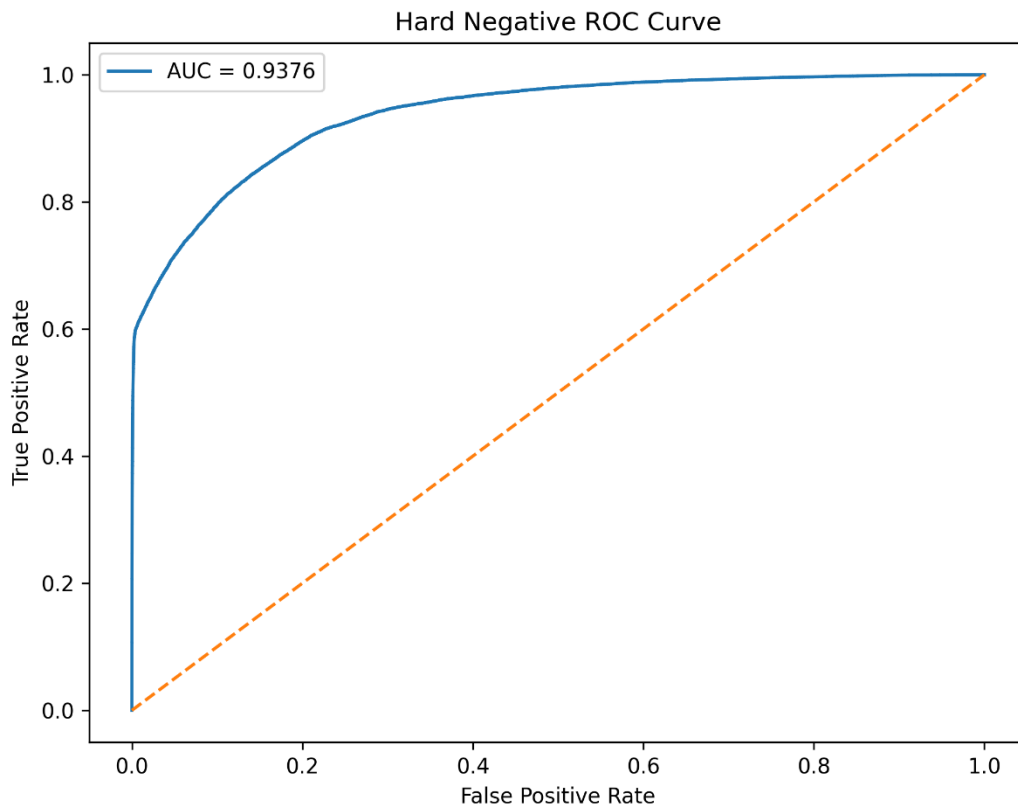


Рисунок 3.3 – ROC-кривая ranking-модели

Высокое значение ROC-AUC свидетельствует о способности модели корректно различать положительные и отрицательные игровые сценарии.

Полученное значение ROC-AUC составило approximately 0.94, что свидетельствует о высоком качестве классификации игровых комбинаций.

Модель демонстрирует устойчивое разделение успешных и неуспешных игровых сценариев даже при наличии сложных hard negative примеров.

Дополнительно выполнялся анализ Precision-Recall зависимости. PR-кривая особенно важна для ranking-задач, поскольку позволяет оценивать качество модели при работе с несбалансированными и сложными выборками.

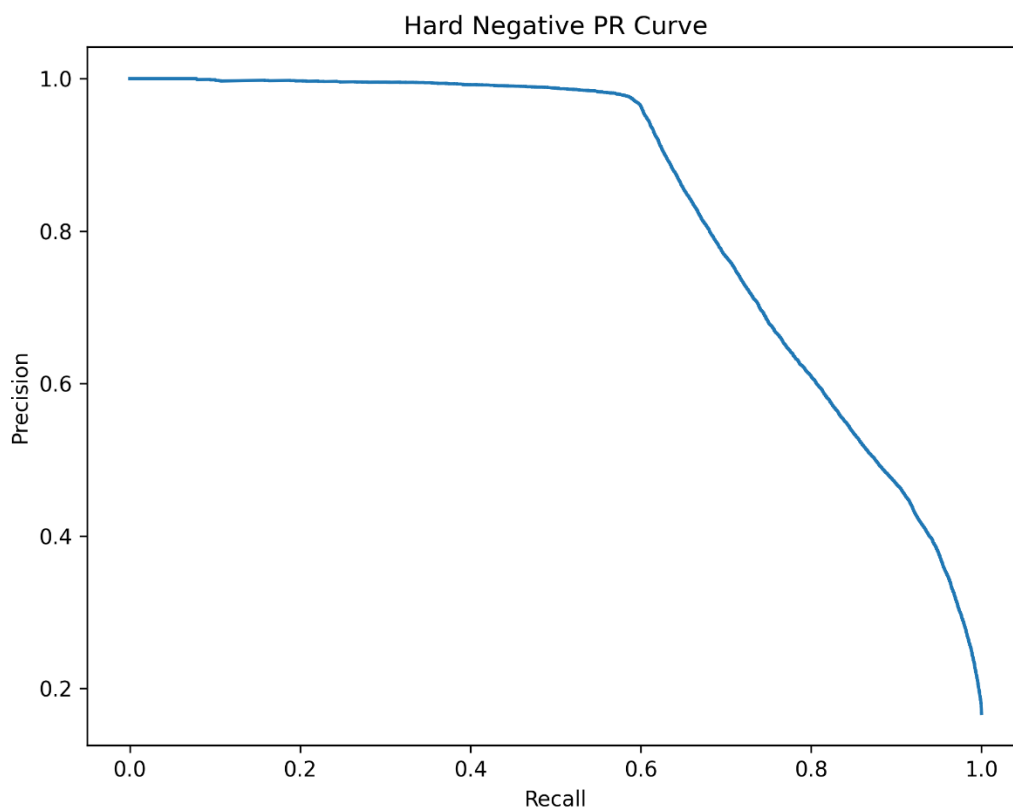


Рисунок 3.4 – Precision-Recall кривая модели

Анализ PR-кривой показывает высокое значение precision на значительной части диапазона recall, что свидетельствует о способности модели формировать качественные рекомендации даже при увеличении количества рассматриваемых кандидатов.

Для дополнительной оценки качества классификации использовалась confusion matrix. Матрица ошибок позволяет оценить, количество корректных классификаций, число false positive ошибок, число false



negative ошибок, общую устойчивость модели. Результаты представлены на рисунке 3.5.

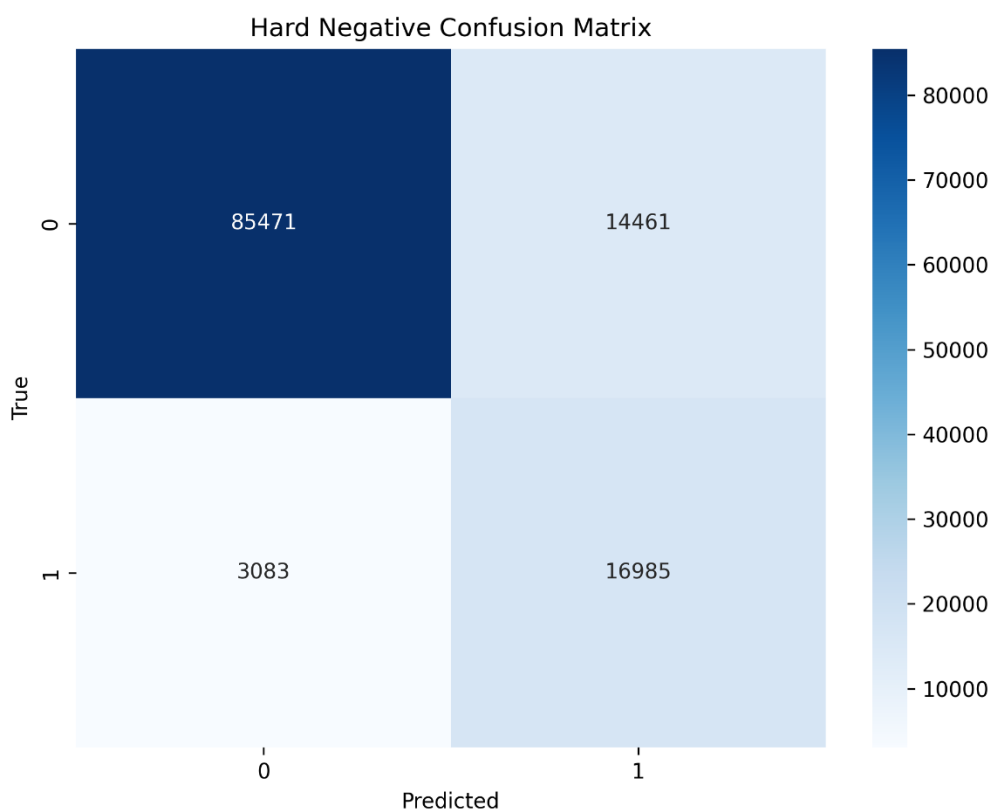


Рисунок 3.5 – Confusion matrix модели

Анализ confusion matrix показывает, что модель демонстрирует высокое качество бинарной классификации игровых сценариев.

Количество корректно классифицированных отрицательных примеров (True Negative) составило 85 471, что свидетельствует о способности модели эффективно отсеивать нерелевантные игровые комбинации и случайных кандидатов.

Число корректно распознанных положительных примеров (True Positive) составило 16 985. Это подтверждает способность модели выявлять эффективные игровые комбинации даже в условиях сложных сценариев.

Количество ошибок False Negative оказалось сравнительно небольшим — 3 083 случаев. Это означает, что модель редко пропускает потенциально хорошие игровые решения.

При этом число False Positive составило 14 461. Наличие подобных ошибок объясняется высокой статистической близостью некоторых игровых персонажей и сложностью задачи ранжирования в условиях большого количества схожих кандидатов. Полученные результаты демонстрируют, что модель обладает:

- высокой чувствительностью к успешным игровым комбинациям
- хорошей способностью различать сложные hard negative примеры
- устойчивостью к статистически близким игровым сценариям

Дополнительно результаты confusion matrix подтверждают эффективность использования: graph-based признаков, embedding-представлений, pairwise learning-to-rank подхода, hard negative sampling.

Таким образом, разработанная модель обеспечивает качественное разделение эффективных и неэффективных игровых комбинаций и может использоваться в качестве основы интеллектуальной рекомендательной системы подбора героев Dota 2.

### **3.5. Разработка пользовательского веб-интерфейса**

Для обеспечения практического использования разработанной рекомендательной модели был реализован пользовательский веб-интерфейс, предназначенный для интерактивного подбора игровых персонажей Dota 2 в режиме реального времени.

Основной задачей интерфейса являлось предоставление удобного инструмента, позволяющего:

- формировать текущий игровой драфт
- выбирать союзных и вражеских героев
- задавать игровую роль и линию

- получать рекомендации по выбору персонажей
- визуализировать взаимодействия между героями
- анализировать синергию и контрпики

В качестве технологической основы веб-приложения использовался фреймворк Streamlit, обеспечивающий быстрое развертывание ML-oriented интерфейсов и интеграцию моделей машинного обучения.

Архитектура приложения была построена по модульному принципу и включала следующие компоненты: frontend-интерфейс взаимодействия с пользователем, recommendation service, feature generation pipeline, ranking inference module, visualization subsystem, graph analytics module.

Структура программной реализации проекта представлена следующими директориями:

- components/
- services/
- utils/
- models/
- data/processed/
- streamlit\_artifacts/

Frontend-часть системы отвечала за:

- отображение интерфейса
- выбор игровых персонажей
- отображение рекомендаций
- визуализацию графов
- отображение статистики героев.

Backend-часть системы обеспечивала:

- загрузку feature-таблиц
- генерацию признаков
- выполнение inference ranking-модели
- вычисление recommendation score

Обученные модели сохранялись в директории:

- models/
- lgbm\_ranker.pkl
- lightgbm\_recommender.pkl
- lightgbm\_hard\_negative.pkl

Интерфейс системы поддерживал интерактивное формирование draft context.

Пользователь мог выбрать:

- союзных персонажей
- вражеских героев
- игровую роль
- игровую линию

После формирования драфта система автоматически запускала inference pipeline и генерировала рекомендации в режиме реального времени. Общая схема взаимодействия компонентов системы представлена на рисунке 3.6.

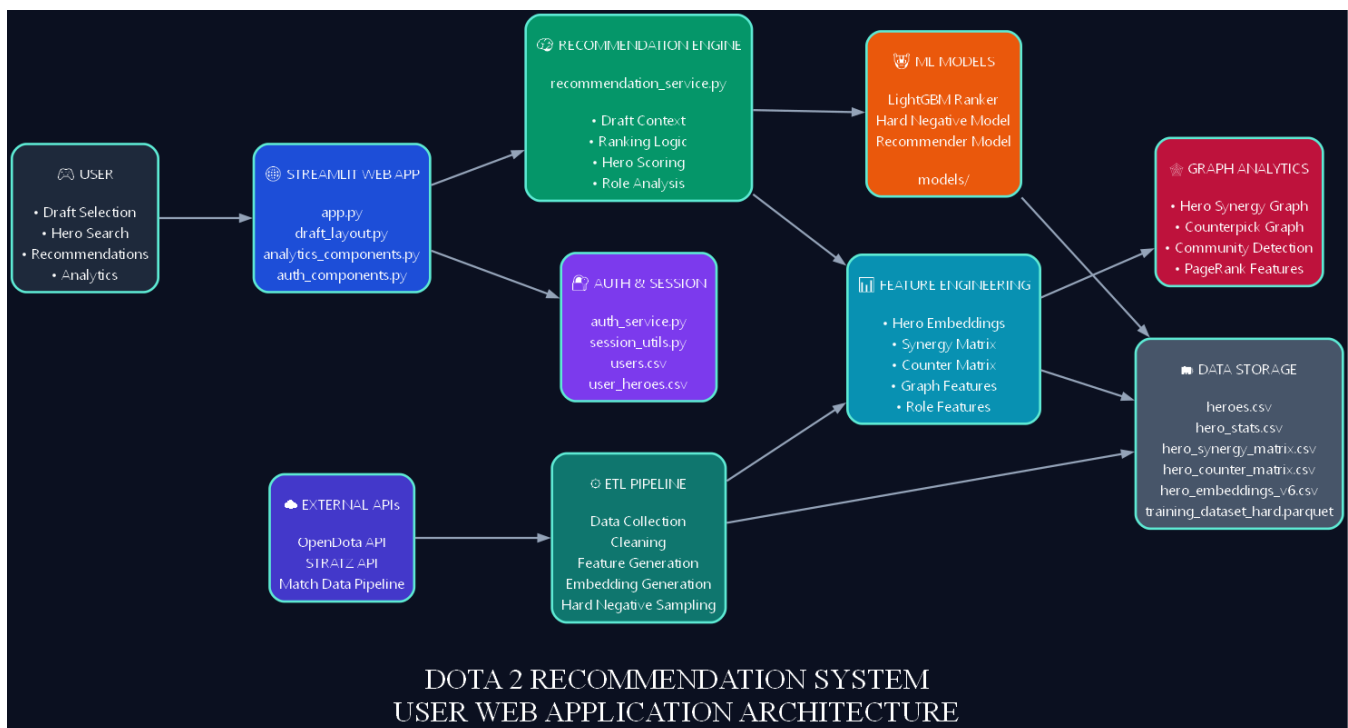


Рисунок 3.6 – Архитектура пользовательского веб-приложения

В процессе работы приложения выполнялись следующие этапы:

1. Загрузка feature-таблиц и обученных моделей
2. Формирование текущего draft context
3. Генерация признаков для кандидатов
4. Построение feature vector
5. Вычисление ranking score
6. Сортировка героев-кандидатов
7. Формирование итоговых рекомендаций

Для каждого героя-кандидата система вычисляла:

- synergy score
- counter score
- embedding similarity
- graph-based признаки
- role compatibility
- итоговый ranking score

Результаты ранжирования отображались пользователю в виде отсортированного списка рекомендуемых персонажей.

Страница регистрации представлена на рисунке 3.7.

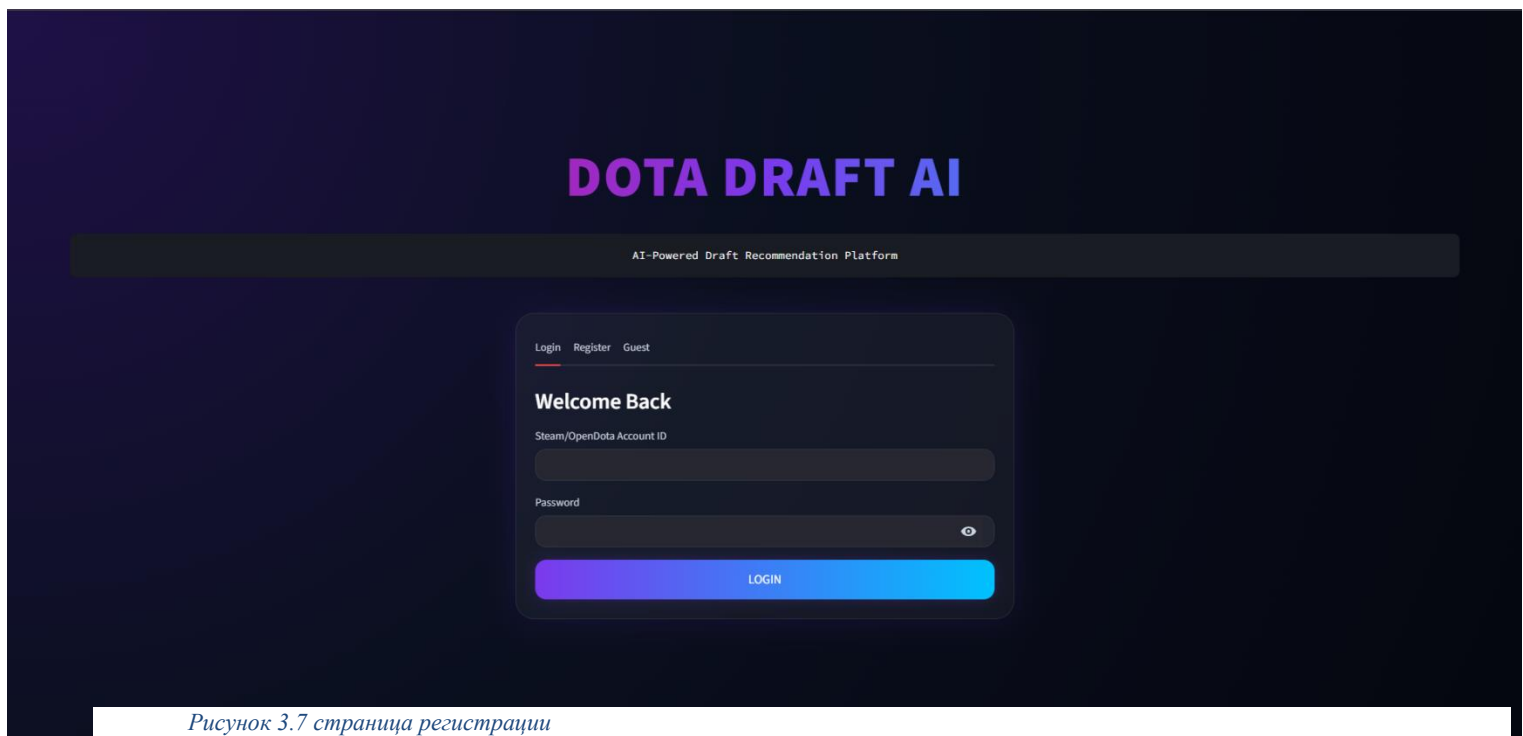


Рисунок 3.7 страница регистрации

Интерфейс выбора игровых персонажей представлен на рисунке 3.8.

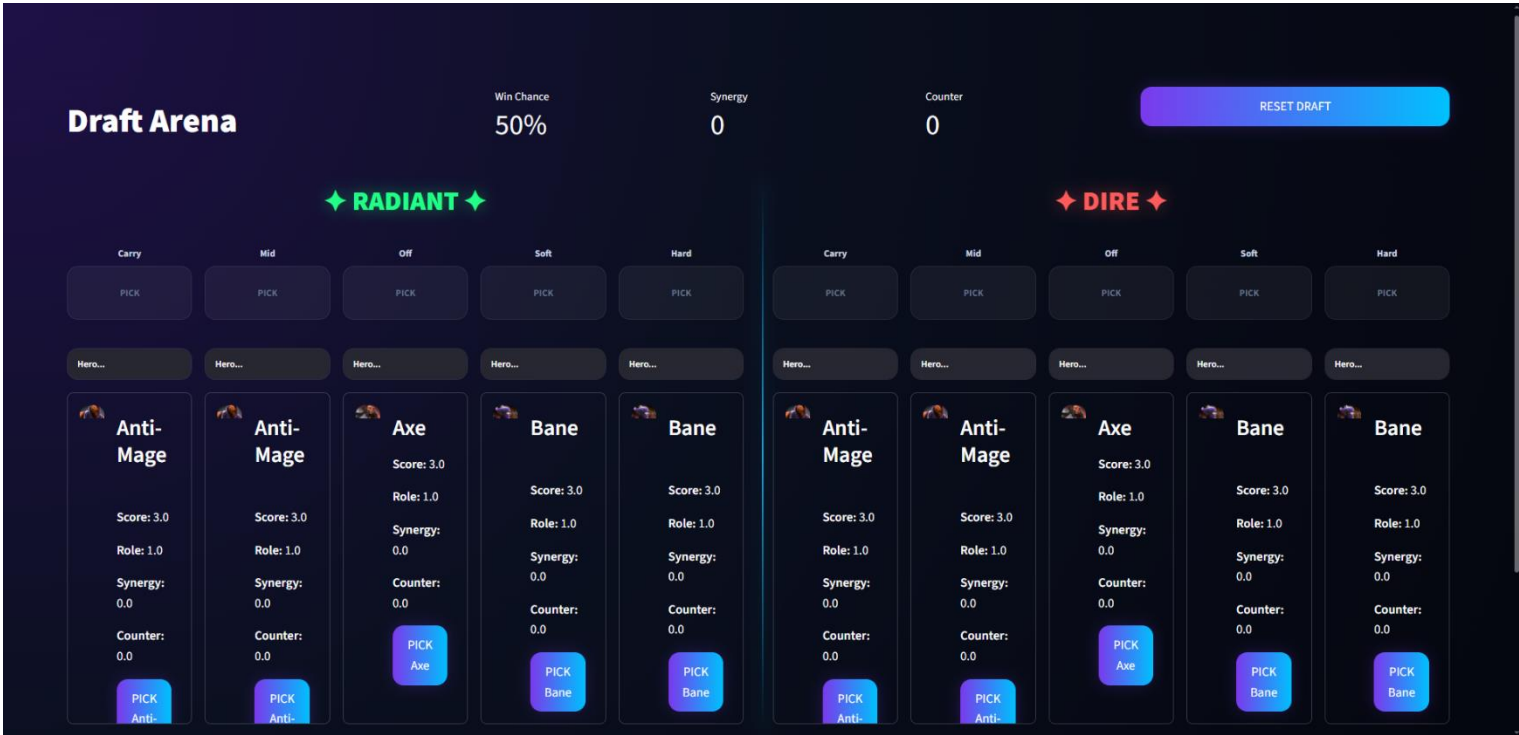


Рисунок 3.8 страница драфтов

Рисунок 3.9 – Генерация рекомендаций игровых персонажей

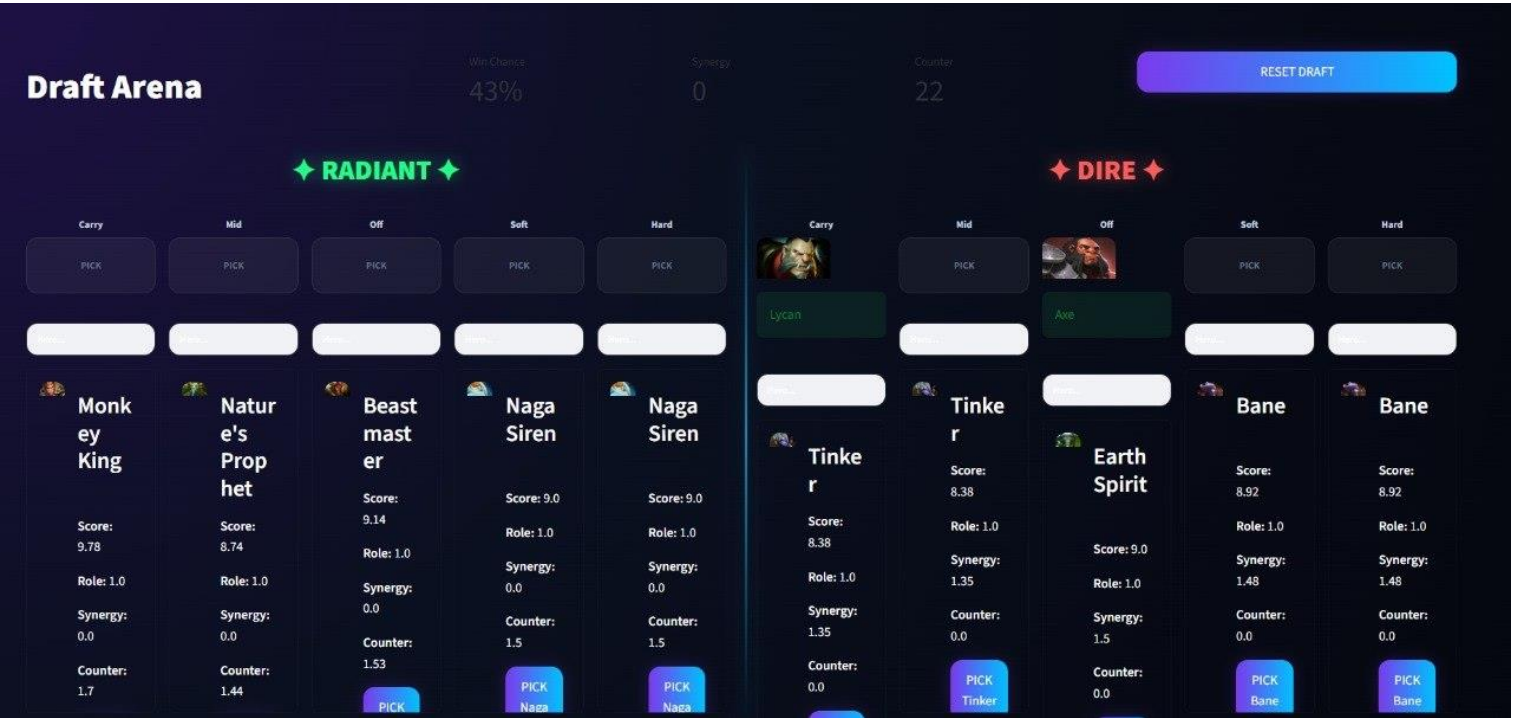


Рисунок 3.9 страница драфтов с выбранными персонажами

Пример визуализации итогового предикта на игру представлен на рисунке 3.10.

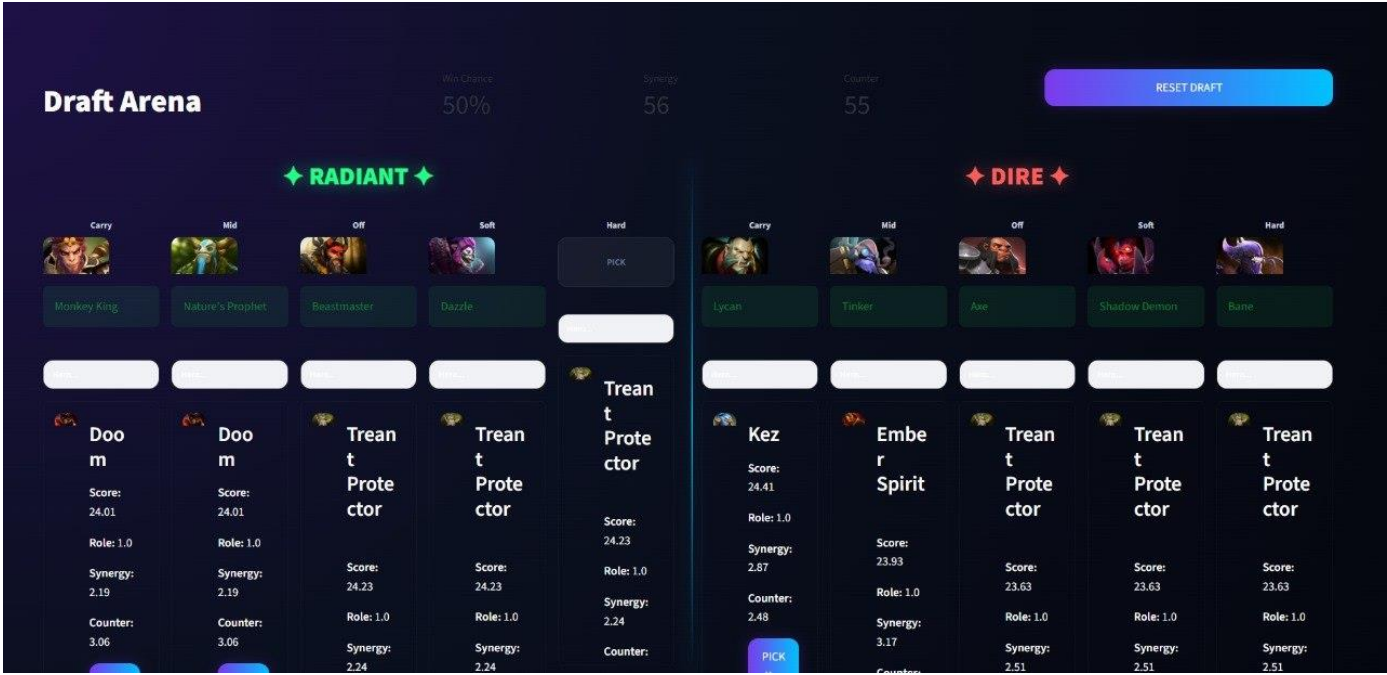


Рисунок 3.10 Пример визуализации итогового предикта на игру

## ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была разработана интеллектуальная рекомендательная система подбора игровых персонажей для Dota 2, основанная на методах машинного обучения, learning-to-rank подходах и графовом анализе данных.

В ходе исследования были рассмотрены современные подходы к построению рекомендательных систем, включая collaborative filtering, content-based filtering и гибридные методы рекомендаций. Особое внимание было уделено специфике задач рекомендаций в многопользовательских соревновательных играх, где эффективность выбора персонажа определяется не только индивидуальными характеристиками героя, но и сложными взаимодействиями между союзниками, противниками, игровыми ролями и текущим состоянием драфта.

В процессе работы была спроектирована и реализована полноценная архитектура ETL-процесса обработки матчевых данных. Для формирования обучающей выборки использовались OpenDota API и STRATZ API, обеспечивающие доступ к статистике матчей Dota 2. В результате автоматизированного сбора и очистки данных был сформирован игровой датасет, содержащий 881 290 обучающих записей и информацию о 129 игровых персонажах.

Для анализа сложных взаимодействий между героями использовались графовые методы анализа данных. Были построены графы синергий и контрпигов персонажей, а также вычислены признаки PageRank, clustering coefficient, degree centrality и community labels.

Дополнительно была реализована генерация embedding-представлений героев в латентном пространстве признаков размерностью 129 измерений, что позволило учитывать скрытые similarity-зависимости между игровыми персонажами.



Для повышения качества обучения ranking-модели использовался подход *hard negative sampling*, обеспечивающий генерацию сложных отрицательных примеров и улучшение способности модели различать близкие игровые сценарии.

В качестве основной рекомендательной модели был выбран алгоритм *LightGBM Ranker*, реализующий *learning-to-rank* подход к ранжированию игровых персонажей. В ходе экспериментов модель продемонстрировала высокую эффективность при обработке сложных *draft*-сценариев и показала устойчивость к пограничным игровым ситуациям.

В рамках практической части работы было разработано пользовательское веб-приложение для интерактивного подбора игровых персонажей. Реализованная система позволяет:

- формировать рекомендации в режиме реального времени
- анализировать текущий драфт
- учитывать роли и игровые линии
- выполнять ранжирование героев по степени релевантности
- визуализировать результаты рекомендаций

Разработанная система обладает модульной архитектурой, поддерживает масштабирование и может быть адаптирована для других многопользовательских игр с *draft*-механиками.

Практическая значимость работы заключается в возможности применения разработанных методов:

- в игровых аналитических сервисах
- в *recommendation*-платформах
- в системах киберспортивной аналитики
- в интеллектуальных системах поддержки принятия решений

Перспективы дальнейшего развития работы включают:

- использование нейросетевых *ranking*-моделей

- применение Graph Neural Networks
- учет временной динамики игровой меты
- online-learning обновление моделей
- интеграцию realtime draft-анализатора
- расширение системы multi-objective рекомендациями
- внедрение разработанной рекомендательной системы непосредственно в игровой клиент
- добавление интеллектуальной системы рекомендаций игровых предметов с учетом текущего драфта, ролей союзников и состава команды противника
- разработку режима тренировки draft-сценариев против AI-бота для анализа стратегий выбора персонажей
- создание пользовательской рейтинговой системы с системой очков и сравнением эффективности драфтов между игроками
- реализацию персонализированных рекомендаций на основе индивидуального игрового стиля пользователя
- расширение аналитической части веб-приложения визуализацией winrate-статистики, популярных стратегий и динамики изменений меты.

Таким образом, поставленные цели и задачи выпускной квалификационной работы были полностью выполнены, а разработанная рекомендательная система продемонстрировала эффективность применения методов машинного обучения и графового анализа для задач интеллектуального подбора игровых персонажей в Dota 2.

«Данная работа выполнена мной самостоятельно»

Денисов Егор Алексеевич

« \_\_\_\_ » \_\_\_\_\_ 2026 г.

## **СПИСОК ЛИТЕРАТУРЫ (ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ) И ИНТЕРНЕТ-РЕСУРСОВ**

1. Аггарвал, Ч. Рекомендательные системы. Учебник / Ч. Аггарвал. – М.: Вильямс, 2020. – 1120 с.
2. Вьюгин, В. В. Математические основы теории машинного обучения и прогнозирования / В. В. Вьюгин. – М.: МЦНМО, 2013. – 387 с.
3. Рашка, С. Python и машинное обучение: машинное и глубокое обучение с использованием Python, scikit-learn и TensorFlow / С. Рашка, В. Мирджалили. – СПб.: Диалектика, 2020. – 848 с.
4. Флах, П. Машинное обучение. Наука и искусство построения алгоритмов, которые извлекают знания из данных / П. Флах. – М.: ДМК Пресс, 2015. – 400 с.
5. Хайкин, С. Нейронные сети: полный курс / С. Хайкин. – М.: Вильямс, 2006. – 1104 с.
6. Hastie, T. The Elements of Statistical Learning: Data Mining, Inference, and Prediction / T. Hastie, R. Tibshirani, J. Friedman. – New York: Springer, 2009. – 745 p.
7. Ke, G. LightGBM: A Highly Efficient Gradient Boosting Decision Tree / G. Ke [et al.] // Advances in Neural Information Processing Systems 30 (NIPS 2017). – 2017. – P. 3146-3154.
8. Ricci, F. Recommender Systems Handbook / F. Ricci, L. Rokach, B. Shapira. – Springer, 2015. – 1003 p.
9. OpenDota API Documentation [Электронный ресурс]. – URL: <https://docs.opendota.com/> (дата обращения: 12.03.2026).
10. Streamlit – The fastest way to build and share data apps [Электронный ресурс]. – URL: <https://streamlit.io/> (дата обращения: 15.03.2026).
11. LightGBM Documentation: Microsoft Corporation [Электронный ресурс]. – URL: <https://lightgbm.readthedocs.io/> (дата обращения: 18.03.2026).

- 12.Scikit-learn: Machine Learning in Python [Электронный ресурс]. – URL: <https://scikit-learn.org/> (дата обращения: 20.03.2026).
- 13.Pandas documentation: Python Data Analysis Library [Электронный ресурс]. – URL: <https://pandas.pydata.org/docs/> (дата обращения: 21.03.2026).
- 14.Optuna: A hyperparameter optimization framework [Электронный ресурс]. – URL: <https://optuna.org/> (дата обращения: 25.03.2026).
- 15.General Data Protection Regulation (GDPR) – Official Legal Text [Электронный ресурс]. – URL: <https://gdpr-info.eu/> (дата обращения: 02.04.2026).
- 16.Официальный сайт киберспортивной дисциплины Dota 2 [Электронный ресурс]. – URL: <https://www.dota2.com/> (дата обращения: 05.04.2026).
- 17.Bishop, C. M. Pattern Recognition and Machine Learning / C. M. Bishop. – New York: Springer, 2006. – 738 p.
- 18.Goodfellow, I. Deep Learning / I. Goodfellow, Y. Bengio, A. Courville. – Cambridge: MIT Press, 2016. – 775 p.
- 19.Sutton, R. Reinforcement Learning: An Introduction / R. Sutton, A. Barto. – Cambridge: MIT Press, 2018. – 552 p.
- 20.Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow / A. Géron. – Sebastopol: O'Reilly Media, 2022. – 851 p.
- 21.Murphy, K. Machine Learning: A Probabilistic Perspective / K. Murphy. – Cambridge: MIT Press, 2012. – 1104 p.
- 22.Leskovec, J. Mining of Massive Datasets / J. Leskovec, A. Rajaraman, J. Ullman. – Cambridge: Cambridge University Press, 2020. – 626 p.
- 23.Manning, C. Introduction to Information Retrieval / C. Manning, P. Raghavan, H. Schütze. – Cambridge: Cambridge University Press, 2008. – 506 p.

24. NetworkX Documentation [Электронный ресурс]. – URL: <https://networkx.org/documentation/stable/> (дата обращения: 11.04.2026).
25. NumPy Documentation [Электронный ресурс]. – URL: <https://numpy.org/doc/> (дата обращения: 12.04.2026).
26. PyTorch Documentation [Электронный ресурс]. – URL: <https://pytorch.org/docs/stable/index.html> (дата обращения: 15.04.2026).
27. Apache Parquet Documentation [Электронный ресурс]. – URL: <https://parquet.apache.org/docs/> (дата обращения: 17.04.2026).
28. STRATZ API Documentation [Электронный ресурс]. – URL: <https://docs.stratz.com/> (дата обращения: 19.04.2026).
29. Valve Developer Community – Dota 2 API [Электронный ресурс]. – URL: [https://developer.valvesoftware.com/wiki/Dota\\_2\\_Workshop\\_Tools/Scripting/API](https://developer.valvesoftware.com/wiki/Dota_2_Workshop_Tools/Scripting/API) (дата обращения: 22.04.2026).
30. Kaggle: Machine Learning and Data Science Community [Электронный ресурс]. – URL: <https://www.kaggle.com/> (дата обращения: 24.04.2026).

## **ПРИЛОЖЕНИЯ**

<https://github.com/oldwildman/RecSisDota2.git>